Chapter 1: Block Cipher and Data Encryption Standard (DES)

Block Cipher and Data Encryption Standards:

- Block Cipher Principles,
- Data Encryption Standards
- the Strength of DES
- Differential and Linear Crypt Analysis
- Block Cipher Design Principles.

❖ Introduction to Cryptography

Cryptography is the process of protecting information by converting readable data (plaintext) into an unreadable form (cipher text) so that only authorized users can access it.

Example:

Plain Text→HELLO

Encrypted Text → XQPLMZ

Only the receiver possessing the correct **secret key** can decrypt the cipher text back into plaintext.

Cryptography is used to:

- Protect confidential information.
- Secure online banking transactions.
- Protect passwords.
- Secure military communications.
- Secure e-commerce websites.
- Protect cloud data.

❖ Basic Terminology

Term	Meaning
Plaintext	**Original readable message**
Ciphertext	**Encrypted message**
Encryption	**Converting plaintext into ciphertext**
Decryption	**Converting ciphertext back into plaintext**
Cipher	**Algorithm used for encryption**
Key	**Secret value used during encryption/decryption**

Encryption Process

Secret Key



Plain Text → Encryption Algorithm → Cipher Text

Decryption Process

Cipher Text → Decryption Algorithm → Plain Text



Secret Key

❖ Symmetric Key Encryption

In symmetric encryption,

- Same key is used for encryption.
- Same key is used for decryption.

Secret Key
Sender ---------- Receiver
Plaintext → Encryption → Cipher text
Ciphertext → Decryption → Plaintext

Examples:

- DES
- Triple DES
- AES
- RC4

Advantages of Symmetric Encryption

- Fast
- Easy to implement
- Suitable for large data
- Low memory usage

Disadvantages of Symmetric Encryption

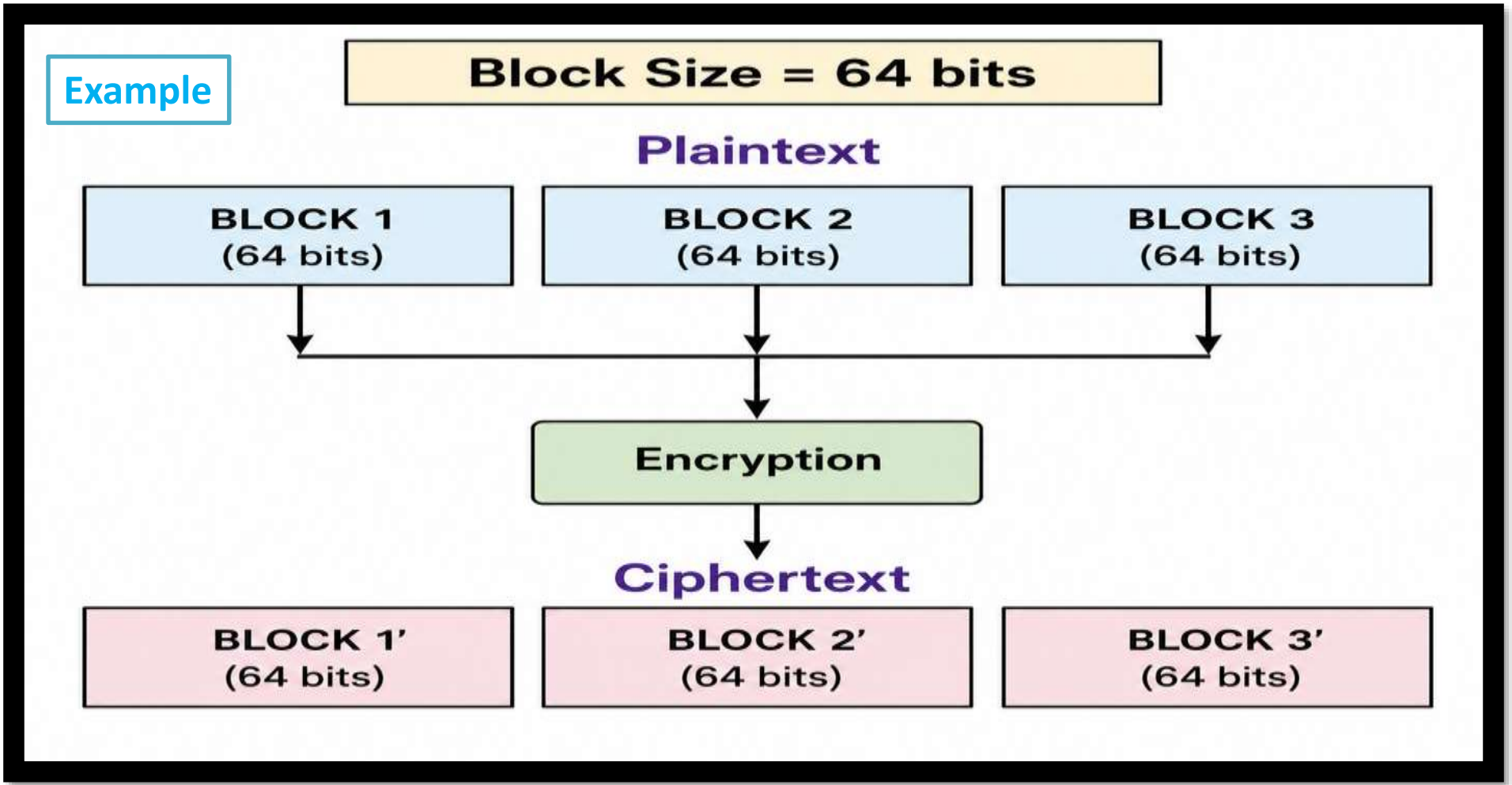
- Key distribution problem
- Same key must be shared securely
- Not suitable for digital signatures

❖ Block Cipher?

A **Block Cipher** is a symmetric encryption algorithm that encrypts **fixed-size blocks of plaintext** into cipher text using the **same secret key**.

Unlike stream ciphers, which encrypt one bit or byte at a time, block ciphers encrypt an entire block simultaneously.

Most modern cryptographic algorithms are block ciphers.



Block Cipher Working(Diagram Explanation)

Step 1: Plaintext is divided into fixed-size blocks.

Example: Plaintext→ COMPUTERSECURITY

|

COMP

UTER

SECU

RITY

Step 2: Each block is encrypted using the same secret key.

Plaintext Block

↓

Encryption

↓

Cipher text Block

Step 3

Cipher text blocks are combined to produce the final cipher text.

Characteristics

- Encrypts fixed-size blocks.
- Uses a secret key.
- Same key is used for encryption and decryption.
- Produces secure cipher text.
- Difficult to guess original data without the key.
- Fast and efficient.

Features of Block Cipher

- Symmetric encryption
- Fixed block length
- Multiple encryption rounds
- Uses substitution and permutation
- High security
- Suitable for large data

Advantages

- High security.
- Strong Encryption
- Suitable for large data.
- Resistant to many attacks.
- Widely accepted.

Disadvantages

- Requires secure key sharing
- Same key for encryption and decryption
- Weak key management reduces security

Applications

- Online Banking
- ATM Transactions
- Digital Payments
- Cloud Storage
- Secure Email
- VPN
- Government Communication
- Military Communication

Examples of Block Ciphers

Algorithm	Block Size	Key Size
DES	64 bits	56 bits
Triple DES	64 bits	112/168 bits
AES	128 bits	128/192/256 bits

❖ **Block Cipher Principles:**

A secure block cipher follows several important principles.

The major principles are:

- Confusion
- Diffusion
- Substitution
- Permutation
- Product Cipher
- Multiple Rounds
- Avalanche Effect

➢ Confusion

Confusion hides the relationship between

- Plaintext
- Cipher text
- Secret Key

Purpose: It makes difficult for an attacker to discover the secret key.

Introduced by **Claude Shannon**.

Implemented using:

- Substitution

Example:

Original

A B C D

↓

Encrypted

Q X M L

Without knowing the substitution rule, recovering the plaintext is difficult.

➢ Diffusion

Diffusion spreads the influence of one plaintext bit over many ciphertext bits.

A small change in the plaintext causes many ciphertext bits to change.

Implemented using:

- Permutation

Example:

Original plaintext (8 bits)→1011001**0**

Encrypted ciphertext→11001011

Change **one bit** in the plaintext→1011001**1**

New cipher text→00110110

Difference B/W Confusion and Diffusion

Confusion	Diffusion
Hides key relationship	**Hides plaintext patterns**
Uses substitution	**Uses permutation**
Provides secrecy	**Provides randomness**

Advantages of Confusion

- Hides key relationship
- Prevents key guessing
- Improves encryption security

Advantages of Diffusion

- Hides statistical patterns
- Prevents frequency analysis
- Improves security

➢ Substitution

Substitution replaces one symbol with another symbol.

Example:

A → Q

B → H

C → X

DES uses **S-Boxes** for substitution.

Advantages:

- ❖ Provides confusion
- ❖ Creates non-linearity
- ❖ Improves resistance to attacks

S-Box (Substitution Box)

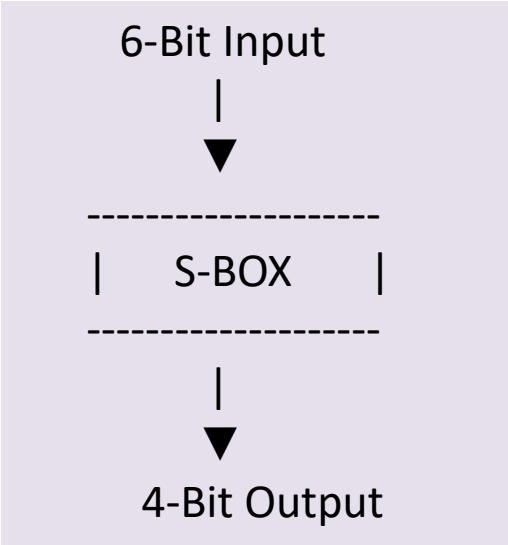
An **S-Box** is a substitution table that converts input bits into different output bits to provide **confusion** in encryption.

S-Box (Substitution Box)

An **S-Box (Substitution Box)** is a table that **replaces (substitutes) input bits with different output bits** according to predefined rules.

It is a **non-linear transformation** used to increase the security of the cipher.

S-Box Diagram



➢ Permutation

Permutation changes only the positions of bits.
The bit values remain unchanged.

Example:
Original→12345678
After Permutation→36571284
DES uses **P-Boxes** for permutation.

Advantages

- Provides diffusion
- Breaks patterns
- Spreads bit changes

P-Box (Permutation Box)

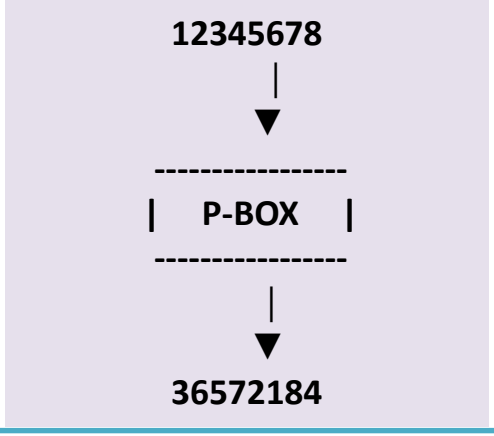
A **P-Box** is a permutation table that rearranges the positions of bits to provide **diffusion**.

P-Box (Permutation Box)

A **P-Box (Permutation Box)** rearranges the **positions of bits** without changing their values.

It spreads the influence of one input bit over many output bits, thereby providing **diffusion**.

P-Box Diagram



Difference Between S-Box and P-Box

S-Box	P-Box
Performs substitution	Performs permutation
Changes bit values	Changes only bit positions
Provides confusion	Provides diffusion
Non-linear operation	Linear rearrangement
Converts 6 bits to 4 bits in DES	Rearranges output bits in DES

➢ Product Cipher

A **Product Cipher** combines substitution and permutation repeatedly.

Plaintext

↓

Substitution

↓

Permutation

↓

Substitution

↓

Permutation

↓

Ciphertext

Example:DES is a Product Cipher.

Advantages

- Strong encryption
- High security
- Better resistance to attacks

➢ Multiple Rounds

Instead of encrypting once, block ciphers repeat the process many times.

Example

DES:

Round 1

↓

Round 2

↓

Round 3

↓

...

↓

Round 16

More rounds generally increase security, though they also increase processing time.

Avalanche Effect

A tiny change in plaintext should produce a huge change in cipher text.

Example
Plaintext →HELLO
Cipher text→ AKMRT
Change only one letter → HELLo
Cipher text becomes→ XQWET

Only one character changed, but the ciphertext changes significantly.

Importance

- Prevents attackers from predicting output.
- Increases security.
- Makes cryptanalysis more difficult.

FEISTEL CIPHER STRUCTURE

The **Feistel Cipher Structure** is a symmetric encryption design developed by **Horst Feistel**. It divides the plaintext into two equal halves and processes them through multiple rounds using different round keys.

Many block ciphers, including **DES**, use the Feistel structure.

Basic Idea:

Plaintext is divided into

- Left Half (L)
- Right Half (R)

Each round performs

- Function F
- XOR
- Swap

Repeated for 16 rounds.

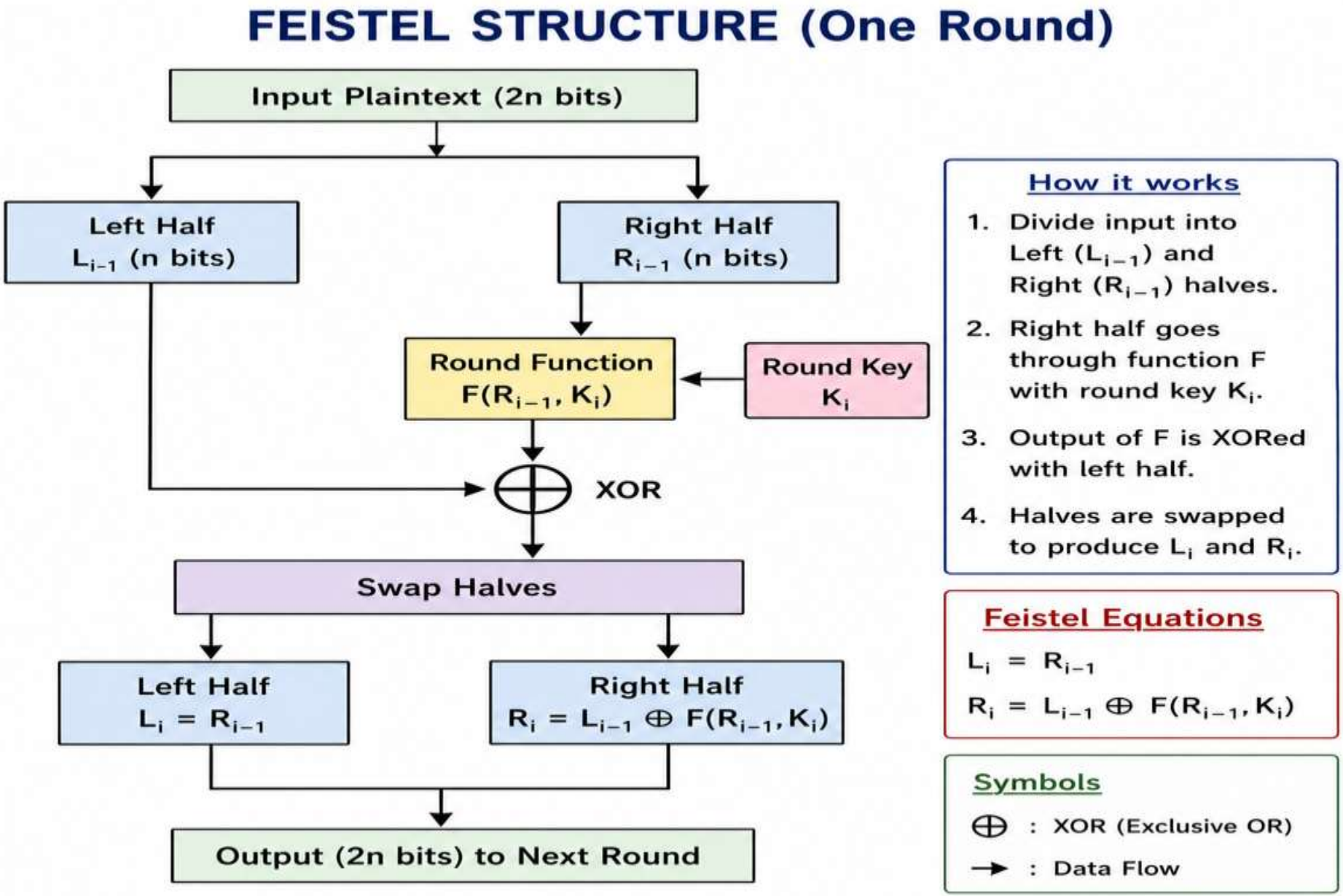
Advantages:

- Same structure is used for encryption and decryption.
- Easy to implement.
- Provides high security through multiple rounds.
- Supports efficient hardware implementation.

Characteristics of Feistel Cipher

- Symmetric cipher
- Multiple rounds
- Uses round keys
- Easy decryption
- High security
- Supports substitution and permutation

Basic Feistel Structure



Feistel Round Formula:

For every round,

Li = Ri-1

Ri = Li-1 XOR F(Ri-1, Ki)

Where,
Li = Left half after the current round
Ri = Right half after the current round
Ki = Round key
F = Round function

Why Swapping?

Swapping ensures:

- Better diffusion
- Better confusion
- Increased security
- Reversible encryption process

The Feistel formula is:
New Left = Old Right
New Right = Old Left XOR F(R,K)

Feistel Encryption Example

Example:1

Suppose 10101100

Left = 1010

Right = 1100

Function output

F(R,K)=0110

Now 1010

XOR

0110

1100

New values

Left = 1100

Right = 1100

After swapping

Left = 1100

Right = 1100

This process repeats for all rounds.

Example:2

If we take F(R,K) = 0000

Suppose 10101100

Left = 1010

Right = 1100

F(R,K) = 0000

So:

New Left = 1100

Now calculate:

1010

$\oplus$ 0000

1010

Therefore:

New Left = 1100

New Right = 1010

After swap:

Left = 1010

Right = 1100

Data Encryption Standard (DES)

❑ Introduction

The **Data Encryption Standard (DES)** is one of the earliest and most widely used **symmetric key block cipher algorithms**. It was developed by **IBM** in the early 1970s and adopted as the **Federal Information Processing Standard (FIPS PUB 46)** by the U.S. government in **1977**.

DES encrypts **64-bit plaintext** into **64-bit ciphertext** using a **56-bit effective secret key**. It is based on the **Feistel Network** and performs **16 rounds** of encryption.

Although DES is historically important, it is no longer considered sufficiently secure for modern applications because of its short key length.

❑ Definition

Data Encryption Standard (DES) is a **symmetric-key block cipher** that encrypts a **64-bit block of plaintext** into a **64-bit block of ciphertext** using a **56-bit effective secret key** and **16 Feistel rounds**.

❑ Features of DES

- Symmetric-key encryption algorithm.
- Encrypts fixed-size blocks.
- Block size = **64 bits**.
- Effective key size = **56 bits**.
- Uses **16 Feistel rounds**.
- Uses substitution and permutation.
- Same algorithm is used for encryption and decryption.
- Fast in hardware implementation.

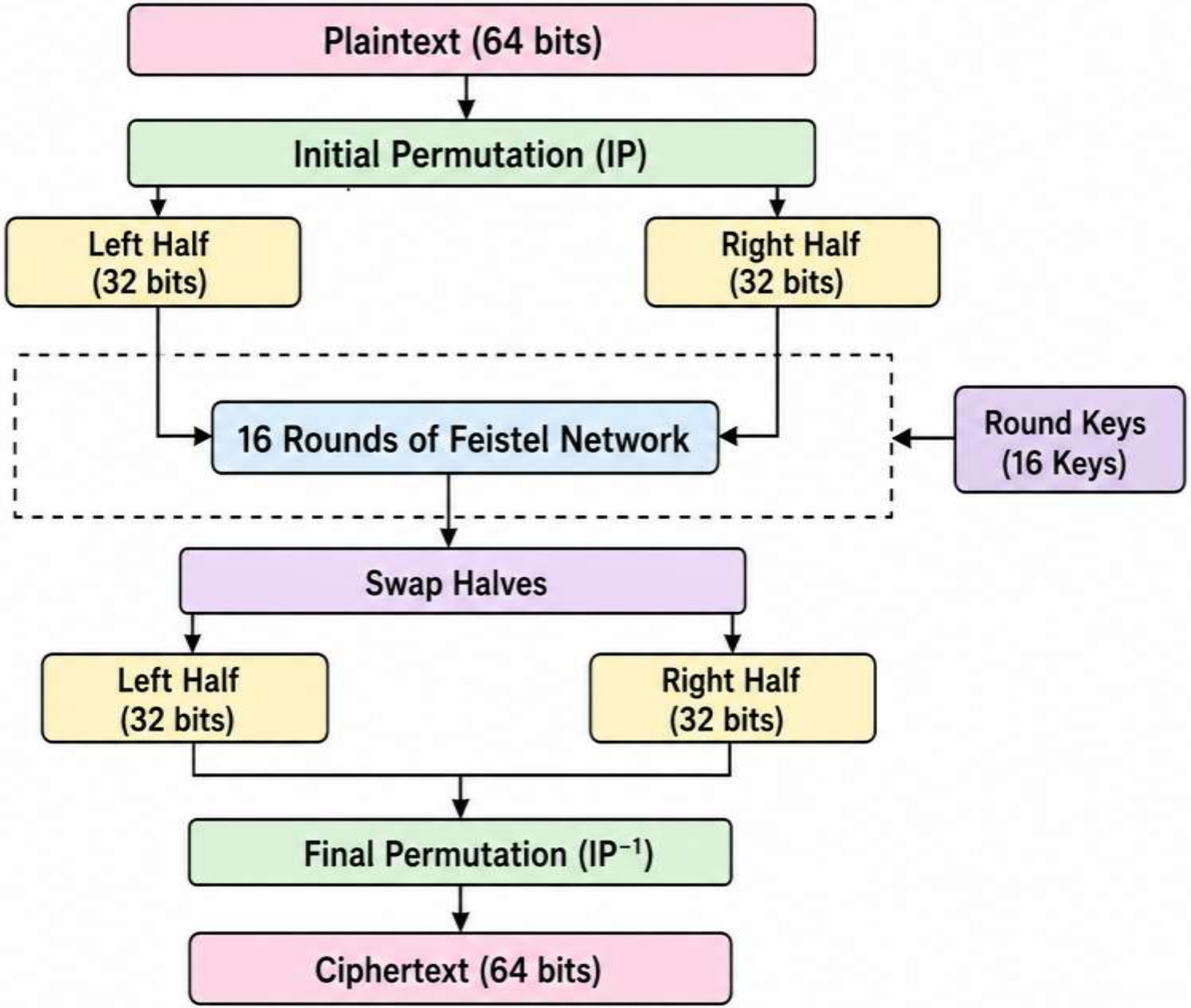
❑ Characteristics of DES

- Based on the **Feistel structure**.
- Uses **16 different round keys**.
- Includes **Initial Permutation (IP)** and **Final Permutation (IP^{-1})**.
- Uses **8 S-Boxes** for substitution.
- Uses a **P-Box** for permutation.
- Provides confusion and diffusion.

DES Encryption Steps—(Summary)

1. Accept **64-bit plaintext**.
2. Apply **Initial Permutation (IP)**.
3. Split into **Left (L0)** and **Right (R0)** of 32 bits each.
4. Perform **16 Feistel rounds** using different round keys.
5. Swap the left and right halves.
6. Apply **Final Permutation (IP^{-1})**.
7. Produce **64-bit ciphertext**.

BLOCK DIAGRAM OF DES



DES Encryption Process

The encryption process consists of the following steps:

Step 1: Input Plaintext

A **64-bit plaintext block** is taken as input.

Example: Plaintext = COMPUTER

Step 2: Initial Permutation (IP)

The 64-bit plaintext undergoes an **Initial Permutation (IP)**.

Purpose:

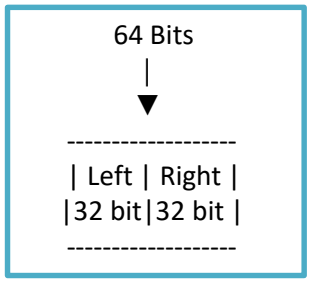
- Rearranges the positions of bits.
- Does not change the bit values.
- Prepares the data for the Feistel rounds.

Step 3: Divide into Two Halves

The permuted block is divided into:

Left Half ($\mathbf{L_o}$) = 32 bits

Right Half ($\mathbf{R_o}$) = 32 bits



Step 4: Sixteen Feistel Rounds

Each round performs the following operations:

1. Expansion of the right half from **32 bits to 48 bits**.
2. XOR with the **48-bit round key**.
3. Substitution using **8 S-Boxes**.
4. Permutation using the **P-Box**.
5. XOR with the left half.
6. Swap the left and right halves.

This process is repeated **16 times**.

Feistel Round Formula

For each round:

$L_i = R_{i-1}$

$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

Where:

L_i = Left half after round *i*

R_i = Right half after round *i*

K_i = Round key

F = Round function

Step 5: Swap Halves

After the 16th round, the left and right halves are swapped.

Step 6: Final Permutation (IP^{-1})

The swapped block undergoes the **Final Permutation**, which is the inverse of the Initial Permutation.

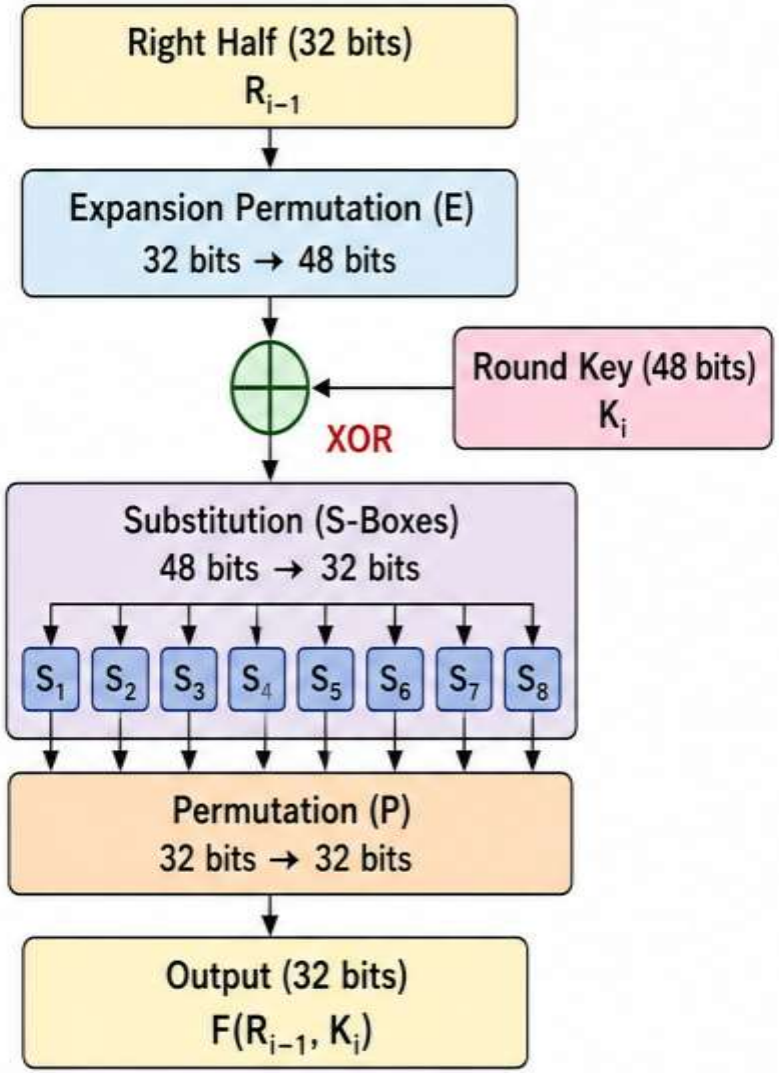
The output is the **64-bit ciphertext**.

DES Round Function (F)-Summary

1. Each round performs:
2. Expansion of 32 bits to 48 bits
3. XOR with the 48-bit round key
4. Substitution using **8 S-Boxes**
5. Permutation using a P-Box

DES ROUND FUNCTION $F(R, K_i)$

Takes 32-bit Right Half and 48-bit Round Key and produces 32-bit output



EXPLANATION (Step by Step)

1. **Expansion Permutation (E)**
   The 32-bit Right Half (R_{i-1}) is expanded to 48 bits using a fixed expansion table. Some bits are repeated to provide diffusion.

2. **XOR with Round Key**
   The 48-bit expanded data is XORed with the 48-bit Round Key (K_i).

3. **Substitution using S-Boxes**
   The 48-bit result is divided into 8 blocks of 6 bits each.
   Each 6-bit block is input to an S-Box (S_1 to S_8).
   Each S-Box produces 4 bits.
   Total output = 8 × 4 = 32 bits.

4. **Permutation (P-Box)**
   The 32-bit output from S-Boxes is permuted using a fixed permutation table (P). This scrambles the bits to provide more diffusion.

5. **Final Output**
   The output is 32 bits, which is used in the Feistel structure as: $L_i = R_{i-1}$
   $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

KEY POINTS

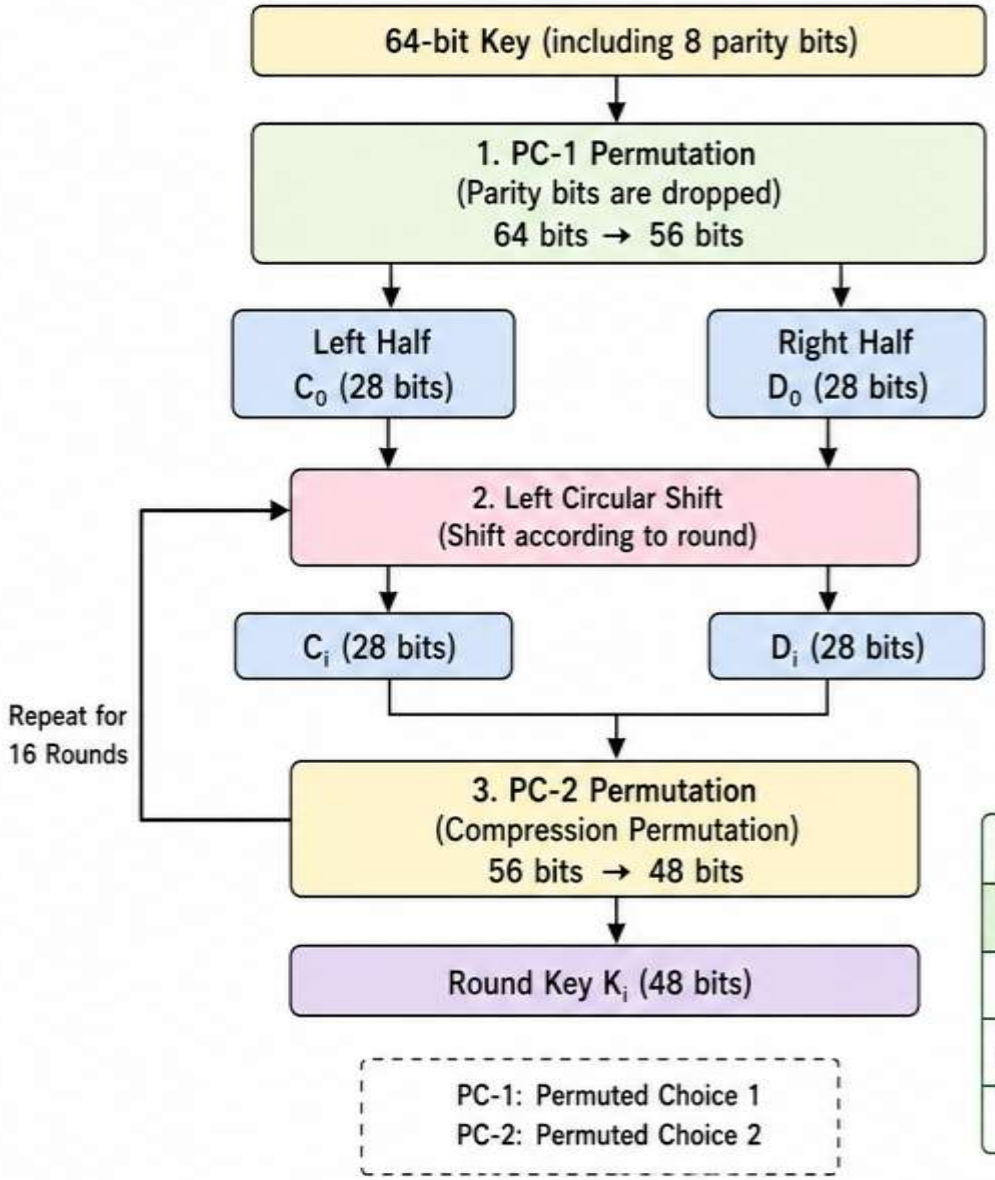
- Provides confusion using S-Boxes.
- Provides diffusion using Expansion and P-Box

SUMMARY OF ROUND FUNCTION

- Input : 32-bit Right Half + 48-bit Round Key
- Process : E → XOR → S-Boxes → P-Box

DES KEY SCHEDULE

Generate 16 Round Keys (K_1 – K_{16}) from 64-bit Key



STEPS IN DES KEY SCHEDULE

1. **PC-1 Permutation:**
   The 64-bit key is permuted using PC-1 table. Parity bits are removed to get 56 bits.
2. **Split:**
   The 56-bit key is split into two halves: C_0 (28 bits) and D_0 (28 bits).
3. **Left Circular Shift:**
   In each round, both halves are shifted left by 1 or 2 bits.
4. **PC-2 Permutation:**
   The shifted 56 bits are permuted using PC-2 table to get 48-bit round key.
5. Repeat steps 2 to 4 for 16 rounds to generate 16 round keys.

LEFT SHIFTS PER ROUND

Round	Left Shifts
1, 2, 9, 16	1 bit
3, 4, 5, 6, 11, 12	2 bits
7, 8, 13, 14, 15	1 bit

OUTPUT

16 Round Keys

K_1
K_2
K_3
.
.
.
.
K_{16}

(Each 48 bits)

DES Key Generation

DES uses a **64-bit input key**.

Out of these:

- **56 bits** are used for encryption.
- **8 parity bits** are discarded.

The 56-bit key is divided into two **28-bit halves**.

Each round performs:

- Left circular shift.
- Compression permutation.

This generates **16 different 48-bit round keys**.

DES Key Generation(Summary)

- Original key = **64 bits**
- Remove 8 parity bits → **56-bit key**
- Split into two 28-bit halves
- Perform left shifts
- Compression permutation generates a **48-bit round key**
- Repeat to obtain **16 round keys**

Why are Parity Bits Used?

Parity bits help detect accidental errors during transmission.

Suppose the key is being sent from one computer to another.

If one bit changes due to noise or a hardware problem, the parity bit helps detect that an error has occurred.

Important: Parity bits **only detect errors**. They **cannot correct errors**, and they are **not used in the encryption calculations**.

DES Decryption

DES decryption uses the **same algorithm** as encryption.

The only difference is the **order of the round keys**.

Encryption→K_1 → K_2 → K_3 → … → K_{16}

Decryption→K_{16} → K_{15} → K_{14} → … → K_1

DES Encryption vs Decryption

Encryption	Decryption
Uses round keys K1 to K16	Uses round keys K16 to K1
Same algorithm	Same algorithm
Different key order	Reverse key order

Applications of DES

- Banking systems (historically)
- ATM machines
- Secure file encryption
- Government communication
- Legacy communication systems

Advantages of DES

- Easy to implement.
- Fast in hardware.
- Uses multiple encryption rounds.
- Strong confusion and diffusion.
- Same structure for encryption and decryption.

Disadvantages of DES

- Small **56-bit key**.
- Vulnerable to brute-force attacks.
- No longer secure for modern applications.
- Replaced by **Triple DES** and **AES**.

Strength of DES refers to the ability of the Data Encryption Standard to protect encrypted information from unauthorized users and resist cryptographic attacks.

The major strengths of DES are:

1. Large Key Space
2. 16 Feistel Rounds
3. Strong S-Boxes
4. Confusion and Diffusion
5. Avalanche Effect
6. Complex Key Schedule
7. Product Cipher Design
8. Symmetric encryption
9. Fast hardware implementation
10. Easy decryption using reverse keys

Weaknesses of DES

- Small 56-bit key
- Vulnerable to brute-force attacks
- Weak and semi-weak keys
- Replaced by stronger algorithms

1. Large Key Space

DES uses a **56-bit effective secret key**.

The number of possible keys is:

2^{56}=72,057,594,037,927,936

Approximately:

72 Quadrillion Keys

When DES was introduced, trying every possible key was practically impossible.

2. Sixteen Feistel Rounds

DES performs **16 rounds** of encryption.

Each round performs:

- Expansion
- XOR with round key
- Substitution (S-Boxes)
- Permutation (P-Box)
- Swap

3. Strong S-Boxes

DES uses **8 carefully designed S-Boxes**.

Each S-Box

6 Bits

↓

4 Bits

4. Confusion and Diffusion

Confusion

Confusion hides the relationship between

- Plaintext
- Ciphertext
- Secret Key

Diffusion

Diffusion spreads one plaintext bit over many ciphertext bits.

5. Avalanche Effect

The **Avalanche Effect** means that changing **one bit** in the plaintext or key should change **many bits** of the ciphertext.

6. Complex Key Schedule

DES generates **16 different round keys** from one secret key.

Each round uses a different **48-bit key**.

This prevents every round from using the same key.

7. Product Cipher Design

DES combines

- Substitution
- Permutation

repeatedly.

This combination is called a **Product Cipher**.

8.Symmetric encryption :

Same key is used for both encryption and decryption process.

9.Fast hardware implementation

Because all these major strengths hardware implementation is fast

10.Easy decryption using reverse keys

The Decryption process is also easy, only thing is to reverse the Key order.

Weak Keys

DES contains-**4 Weak Keys**

- These keys produce identical round keys.
- Such keys reduce encryption security.

Semi-Weak Keys

DES also has-**12 Semi-Weak Keys**

- These keys produce pairs of related round keys.
- They are avoided in practical applications.

Why DES Became Weak?

Although DES was secure in 1977, it is no longer secure today.

The main reasons are:

1. Small Key Size

DES uses only

56 bits

Modern computers can search this key space quickly.

2. Faster Computers

Today's computers

- Multi-core processors
- GPUs
- Cloud computing
- FPGA hardware

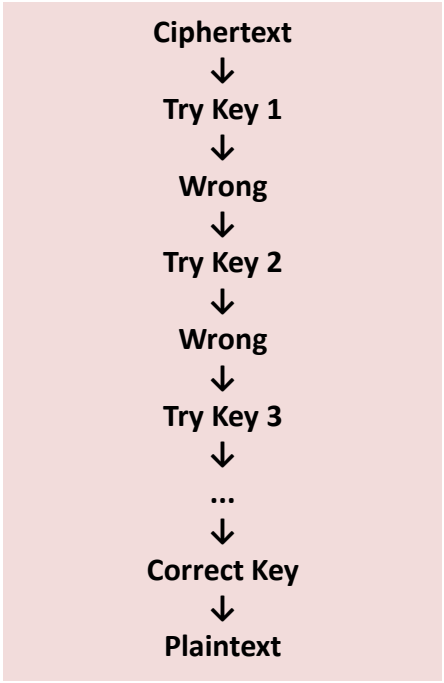
can test millions or billions of keys every second.

3.Brute Force Attack

Definition

A **Brute Force Attack** tries every possible key until the correct key is found.

Brute Force Process



Example

Actual key→ **4587**

Attacker tries

0001

0002

0003

...

4587 ✓

Differential and Linear Cryptanalysis

Cryptanalysis is the study of analyzing encrypted data to discover the plaintext or the secret key without knowing the key.

Objectives of Cryptanalysis

The main objectives are:

- Recover the secret key.
- Recover the original plaintext.
- Identify weaknesses in encryption algorithms.
- Evaluate the security of cryptographic systems.

Types of Cryptanalysis

The common types are:

- Ciphertext-only attack
- Known-plaintext attack
- Chosen-plaintext attack
- Chosen-ciphertext attack
- Differential cryptanalysis
- Linear cryptanalysis

Differential Cryptanalysis

Differential Cryptanalysis is a **chosen-plaintext attack** that analyzes how specific differences between two plaintexts affect the differences in the corresponding cipher texts to obtain information about the secret key.

Basic Idea

- Instead of encrypting a single plaintext, the attacker encrypts **two carefully chosen plaintexts**.
- The attacker then compares the cipher texts and studies how the input difference propagates through the encryption rounds.

Example

Suppose two plaintext blocks are:

P_1 = 11001010

P_2 = 11001110

Step 1: Compute the Input Difference

11001010

XOR

11001110

––––––––

00000100

Difference: ΔP = 00000100

Step 2: Encrypt Both Plaintexts

P_1 → Encryption → C_1

P_2 → Encryption → C_2

Suppose:

C_1 = 10101101

C_2 = 10111101

Step 3: Compute the Output Difference

10101101

XOR

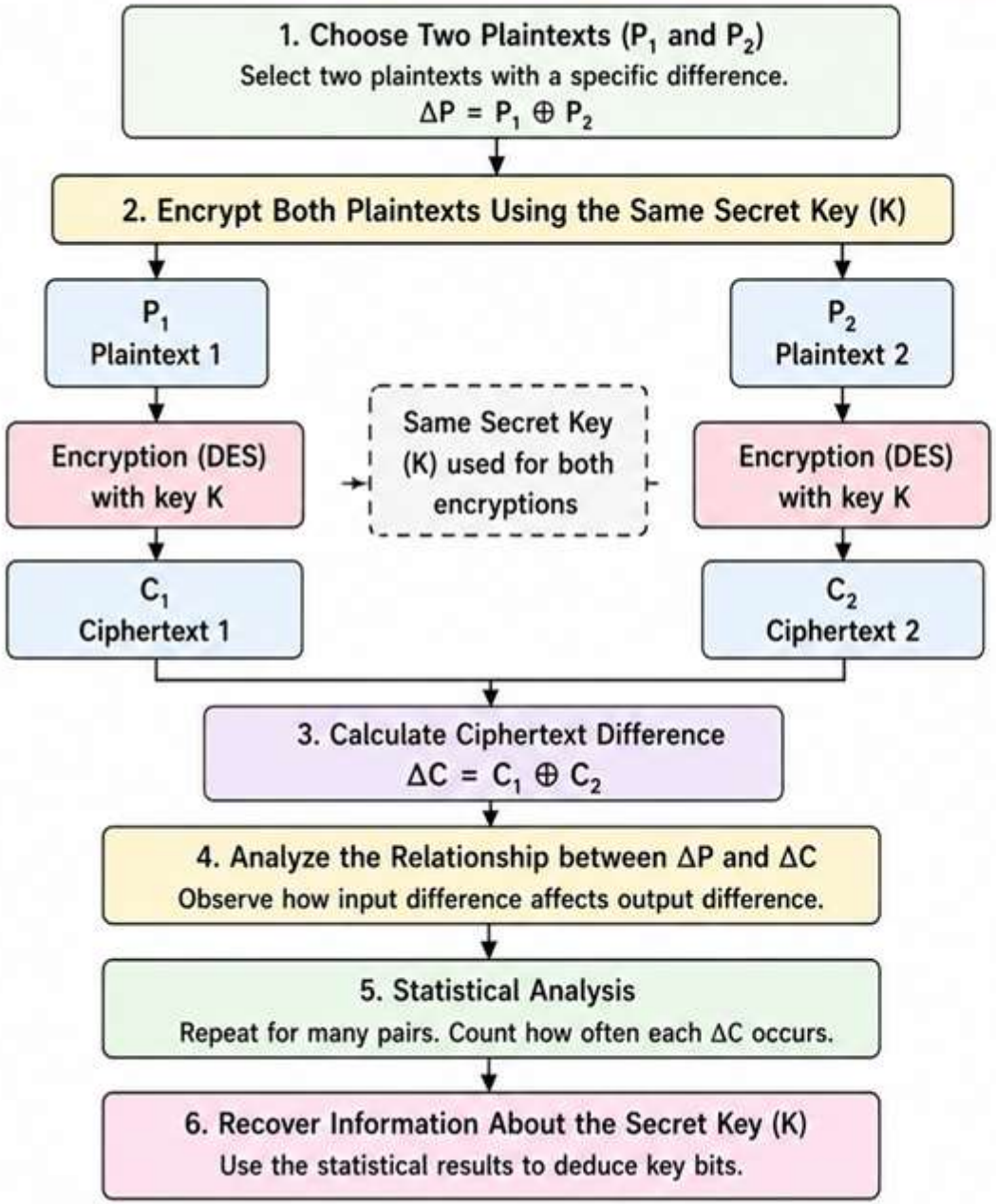
10111101

00010000

ΔC = 00010000

The attacker observes how the input difference (**ΔP**) produces the output difference (**ΔC**). By analyzing many such pairs, information about the round keys may be inferred

DIFFERENTIAL CRYPTANALYSIS PROCESS



EXPLANATION (Step by Step)

1. **Choose Two Plaintexts**
   The attacker chooses two plaintexts (P_1, P_2) with a specific difference ΔP.
2. **Encrypt Both Using Same Key**
   Both plaintexts are encrypted using the same secret key K to get ciphertexts C_1 and C_2.
3. **Calculate Ciphertext Difference**
   Find ΔC by XORing the two ciphertexts.
4. **Analyze ΔP → ΔC Relationship**
   The attacker studies how ΔP is transformed into ΔC through the cipher (rounds, S-boxes, permutations).
5. **Statistical Analysis**
   Repeat the process for many plaintext pairs. Some ΔC values will occur more frequently than others.
6. **Recover Key Information**
   Use the patterns and probabilities to deduce information about the secret key K.

SYMBOLS

P_1, P_2	: Plaintexts
C_1, C_2	: Ciphertexts
ΔP	: Input Difference = $P_1 \oplus P_2$
ΔC	: Output Difference = $C_1 \oplus C_2$
⊕	: XOR (Exclusive OR)
K	: Secret Key

Why Differential Cryptanalysis Works

During DES encryption:

- Differences pass through expansion.
- Differences affect S-Boxes.
- Some output differences occur more frequently than others.
- Statistical analysis helps estimate key bits.

Characteristics:

- Chosen-plaintext attack.
- Uses pairs of plaintexts.
- Relies on XOR differences.
- Uses statistical analysis.
- Very effective against weak cipher designs.

Advantages

- Powerful attack on poorly designed block ciphers.
- Helps evaluate cipher strength.
- Reveals weaknesses in S-Box design.

Limitations

- Requires a large number of chosen plaintext pairs.
- Difficult to perform in real-world systems where chosen plaintext access is unavailable.
- DES was specifically designed to resist practical differential attacks.

Linear Cryptanalysis

Linear Cryptanalysis is a **known-plaintext attack** that uses linear approximations and statistical analysis to discover information about the secret key.

Basic Idea

The attacker collects many pairs of:

- Plaintext
- Corresponding cipher text

Then the attacker searches for equations that are approximately true more often than random chance.

Linear Cryptanalysis Process

Collect Known Plaintext–Ciphertext Pairs



Create Linear Approximations



Count Statistical Bias



Estimate Round Key Bits



Recover Secret Key

Simplified Example:

Suppose the following approximation is tested:

$P1 \oplus P3 \oplus C2 = K5$

Where:

- **P1, P3** = Plaintext bits
- **C2** = Ciphertext bit
- **K5** = Key bit

The attacker evaluates this equation over many known plaintext–cipher text pairs. If it is true significantly more than 50% of the time, it reveals statistical information about **K5**.

Why Linear Cryptanalysis Works

- Perfectly random ciphers should not exhibit predictable linear relationships.
- If such relationships occur with measurable bias, attackers can exploit them to recover key information.

Characteristics

- Known-plaintext attack.
- Uses many plaintext–ciphert ext pairs.
- Employs linear approximations.
- Relies on statistical analysis.
- Effective against weak cipher designs.

Advantages

- Requires only known plaintexts.
- Useful for evaluating cipher security.
- Effective against weak S-Boxes.

Limitations

- Requires a very large number of known plaintexts.
- Computationally intensive.
- Well-designed ciphers such as DES and AES reduce its effectiveness.

Differential vs Linear Cryptanalysis

Feature	Differential Cryptanalysis	Linear Cryptanalysis
Introduced By	Eli Biham & Adi Shamir	Mitsuru Matsui
Attack Type	Chosen-Plaintext Attack	Known-Plaintext Attack
Uses	XOR Differences	Linear Approximations
Input Requirement	Chosen plaintext pairs	Known plaintext–ciphertext pairs
Main Technique	Difference propagation	Statistical bias
Target	Round keys	Round keys
Complexity	High	High

Comparison of DES Attacks

Attack	Technique	Difficulty
Brute Force	Tries every key	Medium (today feasible)
Differential	Difference analysis	High
Linear	Statistical analysis	High

Block Cipher Design Principles

Block Cipher Design Principles are the rules and techniques used to develop a secure and efficient block cipher that provides confidentiality and resists cryptographic attacks.

Objectives of Block Cipher Design

- Provide data confidentiality.
- Resist cryptanalytic attacks.
- Produce random-looking ciphertext.
- Achieve confusion and diffusion.
- Ensure efficient implementation.
- Support secure key management.

Principles of Block Cipher Design

A secure block cipher should satisfy the following principles:

1. Block Size
2. Key Size
3. Number of Rounds
4. Confusion
5. Diffusion
6. Substitution
7. Permutation
8. Product Cipher
9. Avalanche Effect
10. Completeness
11. S-Box Design
12. P-Box Design
13. Feistel Structure
14. Efficient Key schedule
15. Performance and Efficiency

1. Block Size

Definition

Block size is the number of bits encrypted at one time.

Common Block Sizes

Algorithm	Block Size
DES	64 bits
Triple DES	64 bits
AES	128 bits

Importance

- Larger block sizes improve security.
- Reduce repeated ciphertext patterns.
- Better protection against statistical attacks.

Advantages

- Higher security
- Better randomness
- Less pattern repetition

2. Key Size

Definition

Key size is the number of bits in the secret encryption key.

Examples

Algorithm	Key Size
DES	56 bits
Triple DES	112/168 bits
AES	128, 192, 256 bits

Importance

A larger key size:

- Increases the number of possible keys.
- Makes brute-force attacks more difficult.
- Improves overall security.

Example

DES

56-bit Key

↓

2^56 possible keys

3. Number of Rounds

Definition:A **round** is one repetition of encryption operations.

DES uses:**16 rounds**

AES uses:

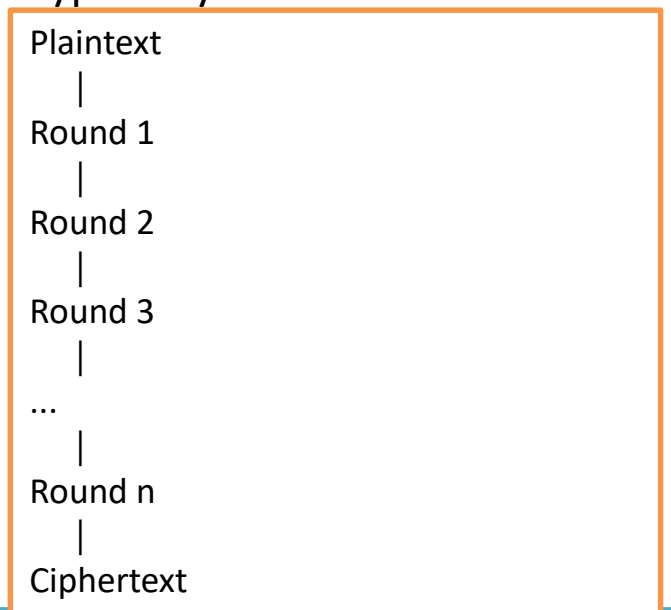
- 10 rounds (128-bit key)
- 12 rounds (192-bit key)
- 14 rounds (256-bit key)

Importance

More rounds:

- Increase confusion.
- Increase diffusion.
- Improve security.
- Make cryptanalysis more difficult.

Diagram



4. Confusion

Definition

Confusion hides the relationship between:

- Plaintext
- Ciphertext
- Secret key

Introduced by **Claude Shannon**.

Achieved By

S-Boxes (Substitution Boxes)

Importance

- Prevents key guessing.
- Increases encryption complexity.
- Protects against cryptanalysis.

5. Diffusion

Definition

Diffusion spreads the influence of one plaintext bit over many cipher text bits.

Achieved By

P-Boxes (Permutation Boxes)

Importance

- Hides statistical patterns.
- Increases randomness.
- Prevents frequency analysis.

6. Substitution

Substitution replaces one value with another.

Example

A → D
B → F
C → M

DES uses

- S-Boxes

AES uses

- SubBytes

Advantages

- Non-linearity
- Better confusion
- Strong security

7. Permutation

Permutation changes only the position of bits.

Example:

Original→12345678
After permutation→36572184

Advantages

- Provides diffusion
- Breaks plaintext patterns

8. Product Cipher

A Product Cipher combines

- Substitution
- Permutation

Repeated several rounds.

Example

Plaintext
↓
Substitution
↓
Permutation
↓
Substitution
↓
Permutation
↓
Ciphertext

9. Avalanche Effect

Definition

A small change in plaintext or key should produce a large change in ciphertext.

Example

Plaintext-1→HELLO
Ciphertext→AF1234
Plaintext-2→HELMO
Ciphertext→98F7AC

Importance

- Better security
- Prevents pattern detection

10. Completeness

Definition

A cipher has **completeness** when every ciphertext bit depends on many plaintext bits and many key bits.

Importance

- Prevents attackers from linking output bits to individual input bits.
- Increases resistance to cryptanalysis.

11. S-Box Design

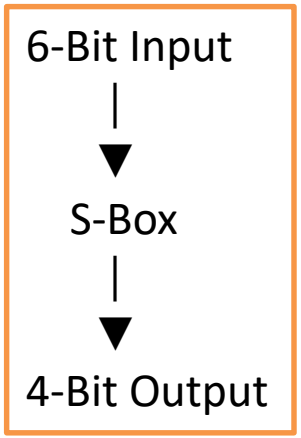
Definition

An **S-Box (Substitution Box)** substitutes input bits with different output bits.

Functions

- Provides confusion.
- Creates non-linear transformation.
- Increases security

Diagram



Characteristics of a Good S-Box

- Non-linear.
- No predictable output.
- Resistant to differential cryptanalysis.
- Resistant to linear cryptanalysis.

12. P-Box Design

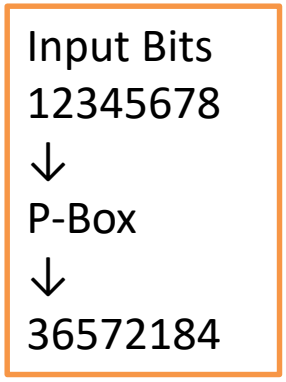
Definition

A **P-Box (Permutation Box)** changes only the positions of bits.

Functions

- Provides diffusion.
- Spreads bit changes.
- Hides statistical information.

Diagram



Characteristics of a Good P-Box

- Efficient bit mixing.
- Uniform distribution.
- Strong diffusion.

13. Efficient Key Schedule

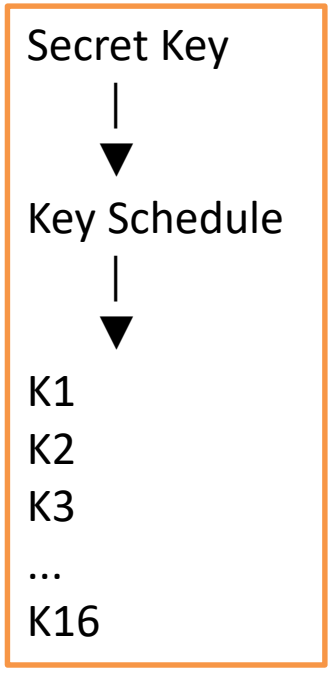
Definition

A **Key Schedule** generates a different round key for each encryption round.

Importance

- Every round uses a different key.
- Prevents repeated key patterns.
- Improves resistance to attacks.

Diagram



14. Feistel Network Design

Most block ciphers use
Feistel Structure.
Each round performs

Split
↓
Function F
↓
XOR
↓
Swap
Repeated several rounds.

Advantages

- Same algorithm for encryption and decryption
- Efficient
- High security

15. Efficiency and Performance

A good block cipher should

- Encrypt quickly
- Decrypt quickly
- Use less memory
- Support hardware
- Support software

Characteristics of a Good Block Cipher

A secure block cipher should have:

- Large block size
- Large key size
- Multiple rounds
- Strong S-Boxes
- Strong P-Boxes
- Avalanche effect
- Completeness
- Efficient key schedule
- Resistance to brute-force attacks
- Resistance to differential cryptanalysis
- Resistance to linear cryptanalysis

Applications

Well-designed block ciphers are used in:

- Online banking
- ATM transactions
- Secure file storage
- VPNs
- SSL/TLS communication
- Digital signatures
- Cloud security

DES vs AES (Design Principles)

Feature	DES	AES
Structure	Feistel Network	Substitution–Permutation Network
Block Size	64 bits	128 bits
Key Size	56 bits	128/192/256 bits
Number of Rounds	16	10/12/14
Confusion	S-Boxes	S-Boxes
Diffusion	P-Boxes	ShiftRows & MixColumns
Security	Lower	Higher

ADVANCED ENCRYPTION STANDARD (AES)

Introduction to AES

The **Advanced Encryption Standard (AES)** is a symmetric-key block cipher developed to replace the older Data Encryption Standard (DES) for a wide range of commercial and government applications. AES was published by the National Institute of Standards and Technology (NIST) in 2001. It encrypts data in fixed blocks of **128 bits** and supports key sizes of **128 bits, 192 bits, and 256 bits**. Unlike DES, AES does not use a Feistel structure. Instead, it processes the entire data block through a sequence of transformations involving byte substitution, permutation, arithmetic over a finite field, and XOR operations with round keys.

The mathematical operations of AES are based on the finite field **GF(2^8)**. Because of its mathematical structure, AES can provide strong security while also being suitable for efficient implementation in both hardware and software. The AES algorithm was selected after an extensive public evaluation process in which different candidate algorithms were examined for security, performance, and implementation characteristics. AES encrypts data quickly and securely and is widely used in banking, online transactions, cloud computing, Wi-Fi security, and government communications.

AES (Advanced Encryption Standard) is a symmetric block cipher that encrypts 128-bit data blocks using 128-bit, 192-bit, or 256-bit keys.

Basic AES Requirements:

- Be a **symmetric block cipher**
- Have a **128-bit block size**
- Support **128-bit, 192-bit, and 256-bit keys**.

AES Selection Process



NIST conducted a public evaluation of proposed algorithms. The final AES standard was published as FIPS PUB 197 in November 2001.

Explanation:

The diagram shows how NIST selected AES from several candidate encryption algorithms:

1. Candidate Algorithms – Different encryption algorithms were submitted for consideration.
2. First Evaluation – NIST tested the algorithms for security, performance, and other criteria.
3. 15 Algorithms – After the first evaluation, 15 candidates were selected for further study.
4. Second Evaluation – The 15 algorithms were analyzed more deeply.
5. 5 Algorithms – Five finalists were selected.
6. Final Evaluation – The five finalists were compared based on security, cost, efficiency, flexibility, and implementation characteristics.
7. Rijndael Selected – Rijndael was chosen as the best candidate.
8. AES Standard – Rijndael became the Advanced Encryption Standard (AES).

EVALUATION CRITERIA FOR AES

Need for a New Encryption Standard

DES was an important encryption standard, but advances in computing technology created concern about its relatively short **56-bit key**. As computer processing power increased, exhaustive key-search attacks became more practical. Therefore, there was a need for a new encryption algorithm that could provide stronger security and support larger key sizes.

NIST initiated a public process to develop a new standard called the **Advanced Encryption Standard**. The purpose was to select an encryption algorithm that would be secure for a long period, efficient in different computing environments, and suitable for widespread use. AES was intended to be a symmetric block cipher and eventually selected to replace DES for many applications.

General Requirements for AES Candidates

The AES competition required candidate algorithms to satisfy certain basic requirements. The algorithm had to be a **symmetric block cipher**. It had to support a block size of **128 bits**. The candidate was also required to support key sizes of **128, 192, and 256 bits**.

The algorithm was expected to be publicly specified and freely available. It should be suitable for implementation in both software and hardware and should not depend on secret design principles. This open evaluation was important because cryptographic strength should be based on the secrecy of the key rather than secrecy of the algorithm itself.

NIST evaluated the AES candidates mainly according to three major categories:

1. **Security**
2. **Cost**
3. **Algorithm and Implementation Characteristics**

These criteria were used to compare the different candidate algorithms and identify the most suitable encryption standard.

Security as an Evaluation Criterion

The most important criterion for AES was **security**. The selected algorithm had to provide strong protection against known cryptanalytic attacks. An attacker should not be able to recover the plaintext or determine the secret key within a practical amount of time.

The algorithm was evaluated against different forms of cryptanalysis. A candidate was considered strong if no attack significantly better than exhaustive key search was known.

For example, if a cipher uses a 128-bit key, the ideal security expectation is that an attacker would require an enormous amount of computational work to search for the correct key.

Key Size and Security

The AES candidates were required to support three key sizes: **128, 192, and 256 bits**. Larger key sizes provide greater resistance to brute-force attacks. The availability of multiple key sizes also allows users to select an appropriate level of security according to their requirements.

A 128-bit key provides a very large key space. A 192-bit or 256-bit key provides even greater protection against exhaustive key searching. Therefore, AES was designed to provide security that would remain effective even as computer technology improved.

The major security criteria were:

1. Actual Security
2. Randomness
3. Soundness
4. Other Security Factors

Actual Security- means how resistant the candidate algorithm is to attacks compared with other candidate algorithms using the same key and block sizes.

NIST compared the actual security of candidate algorithms under equivalent conditions.

A secure algorithm should:

- Resist known attacks.
- Have a sufficient security margin.
- Not reveal useful information about the key.

- Be difficult to cryptanalyze.

Randomness- refers to the extent to which the output of an encryption algorithm is indistinguishable from a random permutation of the input block.

A strong encryption algorithm should produce ciphertext that appears random. Small changes in the plaintext or key should cause large and unpredictable changes in the ciphertext.

This behavior is related to the **avalanche effect**. For example, changing one bit of plaintext should ideally affect many bits of the resulting ciphertext. Similarly, changing one bit of the key should significantly alter the encrypted result. Such properties make it difficult for an attacker to discover useful relationships between input and output.

Soundness- refers to the strength and reliability of the mathematical basis supporting the security of the algorithm.

Other Security Factors - Other security issues identified by the public during the evaluation process.

- Newly discovered attacks
- Weaknesses identified by researchers
- Attacks showing that the actual security was lower than claimed

Resistance to Cryptanalysis

The AES candidate had to resist known cryptanalytic techniques. The design was examined to determine whether weaknesses could be exploited through mathematical analysis of the encryption process.

Particular attention was given to attacks such as **differential cryptanalysis** and **linear cryptanalysis**. Differential cryptanalysis studies how differences between two plaintext inputs affect differences in the corresponding ciphertext outputs. Linear cryptanalysis attempts to find approximate linear relationships between plaintext bits, ciphertext bits, and key bits.

The selected algorithm needed to ensure that its transformations created sufficient **confusion, diffusion, and nonlinearity**, making these attacks impractical.

Security Margin

Another important consideration was the **security margin** of the algorithm. A cipher should not merely resist attacks on its complete number of rounds. Ideally, attacks should also become difficult well before the full number of rounds is reached.

A good security margin provides confidence that the algorithm contains sufficient protection against future improvements in cryptanalysis. If a weakness is discovered in a reduced-round version, the remaining rounds may still provide protection.

Cost as an Evaluation Criterion

The second major evaluation category was **cost**. NIST considered the cost of implementing and using the encryption algorithm.

Cost did not simply mean the monetary price of the algorithm. The evaluation considered the computational resources required to implement it. These resources included memory requirements, processing speed, hardware complexity, software efficiency, and the ability to operate on different types of systems.

The main cost criteria were:

1. Licensing Requirements
2. Computational Efficiency
3. Memory Requirements

Licensing Requirements

Organizations should be able to implement AES without paying restrictive licensing fees.NIST intended AES to be available on a:

- Worldwide basis
- Non-exclusive basis
- Royalty-free basis

It encourages:

- Wide adoption
- Commercial use
- Government use
- Software implementation
- Hardware implementation

Computational Efficiency

An encryption algorithm should perform encryption and decryption efficiently. AES was expected to work effectively on general-purpose computers as well as specialized hardware.

An algorithm that provides excellent security but requires excessive processing time may not be suitable for practical applications. Therefore, performance was an important factor in selecting AES.

Memory Requirements

The amount of memory required by an algorithm was also considered. Some systems have limited memory and cannot efficiently execute algorithms requiring large tables or complex storage structures.

Therefore, an AES candidate needed to have reasonable memory requirements. The algorithm should be suitable for both large computer systems and smaller devices.

Hardware and Software Cost

The algorithm was evaluated for implementation in both **hardware and software**. Hardware implementations are important in devices such as communication equipment and embedded systems, whereas software implementations are necessary for computers and many applications.

An efficient AES algorithm should perform well in both environments. It should not be designed only for one particular type of processor or hardware architecture.

Algorithm and Implementation Characteristics

The third major category concerned the characteristics of the algorithm and its implementation. Even if an algorithm is secure and efficient, its structure should also be practical, understandable, and flexible for different environments.

Flexibility

The algorithm should support different key sizes and operate effectively in different computing environments. AES supports keys of **128, 192, and 256 bits**, providing flexibility for different security requirements.

Simplicity

A simpler algorithm is generally easier to analyze, implement, test, and verify. However, simplicity should not reduce security.

The AES design uses a repeated round structure containing a small number of clearly defined transformations. This makes the overall algorithm systematic while still providing strong cryptographic protection.

Suitability for Different Platforms(Hardware and Software)

The candidate algorithm was evaluated on different platforms. It needed to operate efficiently on small processors, larger processors, hardware devices, and software systems.

This was important because a national encryption standard would be used in many different applications. Therefore, AES had to be practical across a wide range of environments.

Other Final Criteria

Software Implementation Criteria-For software implementations, NIST considered:

- Execution speed
- Performance across different platforms
- Performance changes with different key sizes

Example: An algorithm may be fast with a 128-bit key but slower with a 256-bit key. Therefore, performance at different key sizes is important.

Restricted-Space Environments-Some devices have very limited memory.

Examples:

- Smart cards
- Embedded systems
- Small processors

Requirements- The algorithm should require:

- Small code size
- Low RAM
- Low ROM
- Efficient storage of S-Boxes
- Efficient storage of subkeys

Hardware Implementation-A candidate algorithm should also be efficient in hardware. Hardware can be optimized for:

- Speed
- Size
- Cost

Attacks on Implementations- Security is not limited to mathematical attacks. Some attacks exploit the physical behavior of an implementation.

1. Timing Attacks-An attacker studies the time taken by cryptographic operations**.**

2. Power Analysis-An attacker studies the power consumption of a device during cryptographic operations.

Encryption and Decryption Efficiency

Both encryption and decryption were considered. However, efficient encryption was especially important because certain modes of operation and cryptographic applications use the encryption operation extensively. The design of AES therefore considers implementation efficiency as well as cryptographic strength.

Key Agility- refers to the ability to change cryptographic keys quickly and with minimum resource usage.It includes:

- Fast subkey computation
- Quick switching between keys
- Efficient key setup

Other Versatility and Flexibility:

***Parameter Flexibility*-** The algorithm should allow other key sizes, other block sizes and increasing the number of rounds if future attacks require it.

***Implementation Flexibility*-** The cipher should allow its components to be optimized for particular environments.

Instruction-Level Parallelism- Instruction-Level Parallelism (ILP) means the ability of an algorithm to take advantage of processors that can execute multiple independent instructions efficiently.

So the potential for instruction-level parallelism as one of the evaluation considerations.

Importance

It can improve:

- Processing speed
- CPU utilization
- Overall encryption performance

Selection of Rijndael

After evaluating the candidate algorithms, **Rijndael** was selected as the algorithm for AES. The Rijndael algorithm satisfied the major requirements relating to security, efficiency, flexibility, and implementation.

The selected AES algorithm uses a fixed block size of **128 bits**. It supports key lengths of **128, 192, and 256 bits**. Depending on the key size, AES uses a different number of rounds. For a 128-bit key, AES uses **10 rounds**; for a 192-bit key, it uses **12 rounds**; and for a 256-bit key, it uses **14 rounds**.

- No known security attacks at the time of selection.
- An adequate security margin.
- Good software performance across different platforms.
- Good performance on 8-bit and 64-bit platforms and DSPs.
- Fast key setup.
- High inherent parallelism.
- Low RAM and ROM requirements in restricted-space environments.

Thus:

AES Key Size	Number of Rounds
128 bits	10
192 bits	12
256 bits	14

THE AES CIPHER

AES Block Structure

AES uses: Plaintext → AES Encryption → Ciphertext

Plaintext = 128 bits ,Ciphertext = 128 bits

Secret key = 128, 192, or 256 bits

AES is a: Symmetric + Block + Iterative Cipher

Symmetric-Same secret key is used for encryption and decryption.

Block Cipher -Processes fixed-size blocks of data.

Basic Structure of AES

AES is a **substitution-permutation network** rather than a Feistel cipher. In AES, the complete data block is processed during each round. The 128-bit input as a **4 × 4 matrix of bytes** called the **State**.

The State is arranged as a **4 × 4 matrix of bytes**:

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Since each element is one byte, the matrix contains:

Total = 16 bytes

Each byte = 8 bits

16 × 8 = 128 bits

AES transformations operate on this State matrix.

AES Overall Structure

AES Encryption

Plaintext

↓

Initial AddRoundKey

↓

Round 1

↓

Round 2

↓

..↓

Final Round

↓

Ciphertext

AES-128

- Initial key addition
- 9 normal rounds
- 1 final round
- Total = 10 rounds .

AES Round Structure

Normal AES Round

SubBytes

↓

ShiftRows

↓

MixColumns

↓

AddRoundKey

Final Round

SubBytes

↓

ShiftRows

↓

AddRoundKey

Important-MixColumns is NOT performed in the final round.

AES Round Structure

For AES-128, the input key is expanded into **44 words**, where each word contains 32 bits. Four words, or 128 bits, are used as a round key. These round keys are used during the initial transformation and the ten rounds of AES encryption.

AES uses four main transformations:

1. **SubBytes**
2. **ShiftRows**
3. **MixColumns**
4. **AddRoundKey**

The normal encryption process for AES-128 is:

Initial Round

AddRoundKey

Rounds 1 to 9

SubBytes → ShiftRows → MixColumns → AddRoundKey

Round 10

SubBytes → ShiftRows → AddRoundKey

The final round does not contain the MixColumns transformation.

Simple Flow

```
Plaintext
    ↓
Initial AddRoundKey
    ↓
Round 1 to Round 9
SubBytes
    ↓
ShiftRows
    ↓
MixColumns
    ↓
AddRoundKey
    ↓
Final Round
SubBytes
    ↓
ShiftRows
    ↓
AddRoundKey
    ↓
Ciphertext
```

SubBytes Transformation

The first transformation in an AES encryption round is called **SubBytes**. It performs a substitution on every byte of the State. Each byte of the State is replaced independently. AES uses a fixed **S-box**. The S-box contains **256 entries**. It provides **non-linearity**. It makes the relationship between plaintext, ciphertext and key more complex.

AES uses a special substitution table called the **S-box**. The S-box contains a permutation of all possible byte values. Each input byte is replaced by another byte according to the corresponding value in the S-box. The transformation is therefore performed independently on all 16 bytes of the State. The AES S-box is a **16 × 16 table**.

AES S-Box Lookup Table (Hexadecimal)

AES S-Box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

The row and column values of `53` are used to locate the corresponding entry in the AES S-box. The resulting value replaces `53`.

Thus:

```
Input Byte → AES S-Box → Substituted Byte
```

The main purpose of SubBytes is to introduce **nonlinearity** into AES. Nonlinearity makes it difficult for attackers to express the relationship between plaintext, ciphertext, and key using simple mathematical equations.

AES S-Box Construction

The AES S-Box is constructed in two main steps:

Step 1: Find the multiplicative inverse of the input byte in **GF(2^8)**.

Step 2: Apply an **affine transformation** to the inverse.

The result is the S-Box output.

Formula to remember:

Input→Multiplicative Inverse→Affine Transformation→S-Box Output

Purpose: To provide **nonlinear substitution (confusion)** and strengthen AES against cryptanalysis.

Inverse SubBytes

During decryption, the **inverse S-box** is used. The inverse substitution reverses the effect of the original SubBytes transformation.

Therefore:

```
SubBytes       → Encryption
InvSubBytes    → Decryption
```

ShiftRows Transformation

The second transformation is called **ShiftRows**. This is a permutation operation that changes the positions of bytes in the State.

AES treats the State as four rows. The rows are shifted cyclically to the left by different amounts.

Row 0

The first row is not shifted.

```
a b c d
↓
a b c d
```

Row 1

The second row is shifted left by one byte.

```
a b c d
↓
b c d a
```

Row 2

The third row is shifted left by two bytes.

```
a b c d
↓
c d a b
```

Row 3

The fourth row is shifted left by three bytes.

```
a b c d
↓
d a b c
```

The purpose of ShiftRows is to spread the bytes across different columns. It provides **diffusion** by ensuring that bytes originally located in the same column become distributed into different columns.

Inverse ShiftRows

During decryption, the rows are shifted in the opposite direction.

```
Row 0 → Shift right by 0
Row 1 → Shift right by 1
Row 2 → Shift right by 2
Row 3 → Shift right by 3
```

This reverses the ShiftRows transformation.

MixColumns Transformation

The third transformation is called **MixColumns**. This operation treats each column of the State independently.

Each column contains four bytes. These four bytes are transformed using matrix multiplication over the finite field **GF(2^8)**.

The encryption transformation is represented as:

Matrix

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

one column:

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

After MixColumns:

$$\begin{bmatrix} 02a \oplus 03b \oplus c \oplus d \\ a \oplus 02b \oplus 03c \oplus d \\ a \oplus b \oplus 02c \oplus 03d \\ 03a \oplus b \oplus c \oplus 02d \end{bmatrix}$$

The result becomes the new column of the State.

The purpose of MixColumns is to provide strong diffusion. The bytes within a column are mixed together so that each output byte depends on multiple input bytes.

The combination of **ShiftRows and MixColumns** is particularly important. ShiftRows moves bytes into different columns, while MixColumns mixes the bytes within each column. After several rounds, changes in the input spread throughout the entire State, so that output bits become dependent on many input bits.

Why Are 01, 02 and 03 Used?

The AES designers also considered efficient implementation. Multiplication by the values `{01}`, `{02}`, and `{03}` can be implemented efficiently using shifts and XOR operations. This helps make AES practical for implementation.

Inverse MixColumns

For decryption, an inverse matrix is used:

0E090D0B0B0E090D0D0B0E09090D0B0E

The inverse transformation reverses the mixing performed during encryption.

AddRoundKey Transformation

The fourth transformation is called **AddRoundKey**. This is the transformation in which the secret key directly affects the State.

The current 128-bit State is combined with a 128-bit round key using the bitwise **XOR operation**.

Mathematically:

$$\text{New State} = \text{Current State} \oplus \text{Round Key}$$

Or

$$\text{State}' = \text{State} \oplus \text{RoundKey}$$

Each bit of the State is XORed with the corresponding bit of the round key.

For example:

```
State Byte:      10110110
Round Key Byte:  01101001
                 --------
XOR Result:      11011111
```

The important property of XOR is:

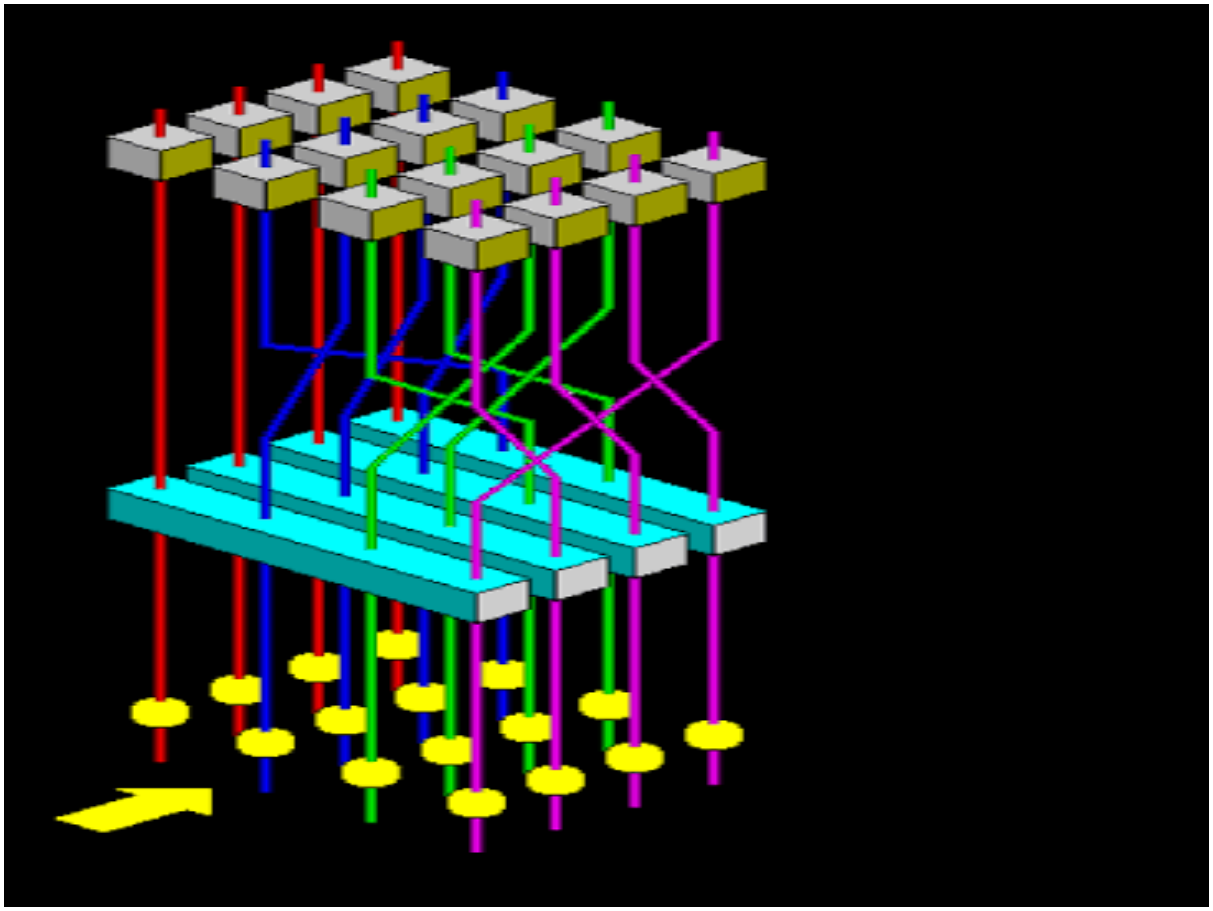
$$A \oplus A = 0$$

and

$$A \oplus 0 = A$$

Therefore, XOR is its own inverse. Applying the same round key again reverses the operation. For this reason, the forward and inverse AddRoundKey transformations are identical. The book describes AddRoundKey as a transformation that affects every bit of the State while remaining very simple to implement.

Only the AddRoundKey transformation directly uses the secret key. The other transformations provide scrambling, confusion, diffusion, and nonlinearity, but do not themselves use the key. AES can therefore be viewed as repeatedly alternating between **key-dependent XOR operations** and **key-independent scrambling operations**.



AES Key Expansion

AES does not use the original cipher key directly in every round. Instead, the original key is expanded into a set of **round keys** using the **Key Expansion Algorithm**, also called the **Key Schedule**.

For AES-128, the original key consists of:

```
128 bits = 16 bytes = 4 words
```

The key expansion algorithm produces:

```
44 words = 176 bytes

Each word = 32 bits
Total round keys = 11
```

These 44 words provide the round keys needed for the initial AddRoundKey operation and the ten encryption rounds. Each round key contains four words.

AES-128 Round Keys

Stage	Round Key Used
Initial AddRoundKey	Round Key 0
Round 1	Round Key 1
Round 2	Round Key 2
Round 3	Round Key 3
Round 4	Round Key 4
Round 5	Round Key 5
Round 6	Round Key 6
Round 7	Round Key 7
Round 8	Round Key 8
Round 9	Round Key 9
Final Round	Round Key 10

Thus, AES-128 requires:

11 round keys×4 words=44 words .

In AES:
1 Word=32 bits=4 bytes
Suppose we have:
W3 = [09 CF 4F 3C]
This is one word because it contains 4 bytes.

Expansion:

AES-128
Key = 128 bits
1 word = 32 bits
128 / 32 = 4 words
Rounds = 10
Initial Round Key = 1
Total Round Keys = 10 + 1
= 11
Each Round Key = 4 words
Total Words = 11 × 4
= 44 words

The Key Expansion Pattern
For AES-128:
W0 W1 W2 W3
↓

W4 W5 W6 W7
↓
W8 W9 W10 W11
↓
...
↓
W40 W41 W42 W43
The important operation is used whenever we generate:
W4, W8, W12, W16, ...

Important Formula:
For the first word of each group:

$Wi = Wi-4 \oplus g(Wi-1)$

where **g()** involves:

- RotWord
- SubWord
- Rcon

For the remaining words:
$Wi = Wi-4 \oplus Wi-1$

Key Expansion Operations:

RotWord- means rotate the bytes of a word by one position to the left.
Suppose:W3 = 09 CF 4F 3C
Apply RotWord:
09 CF 4F 3C
↓
CF 4F 3C 09
Example:
A B C D
↓
B C D A

SubWord-Replace each byte using the AES S-Box.
Suppose after RotWord we have:CF 4F 3C 09
Each byte goes through the S-Box:
CF → S-Box → 8A
4F → S-Box → 84
3C → S-Box → EB
09 → S-Box → 01

Rcon
Rcon = Round Constant
AES uses a different constant for different key-expansion rounds.

For example:

For Round 1:
Rcon[1]=01 00 00 00

Round	Rcon
1	01 00 00 00
2	02 00 00 00
3	04 00 00 00
4	08 00 00 00
5	10 00 00 00
...	...

The key expansion process is designed to provide diffusion and nonlinearity. A change in the original cipher key should affect many bits in the expanded round keys. The design also uses round constants to remove undesirable symmetry between rounds. The book identifies goals such as preventing easy calculation of other round-key bits from partial knowledge, spreading cipher-key differences into round keys, providing sufficient nonlinearity, speed, and simplicity.

Overall AES Encryption Process

For AES-128, the complete process can be summarized as follows.

Step 1: Input

The 128-bit plaintext is placed into the 4 × 4 State matrix.

Step 2: Key Expansion

The original 128-bit cipher key is expanded into 44 words. These words are grouped into 11 round keys.

Step 3: Initial AddRoundKey

The plaintext State is XORed with the first round key.

Step 4: Nine Main Rounds

Each of the next nine rounds performs:

`SubBytes`

```
   ↓
ShiftRows
   ↓
MixColumns
   ↓
AddRoundKey
```

Step 5: Final Round

The tenth round performs:

```
SubBytes
   ↓
ShiftRows
   ↓
AddRoundKey
```

The MixColumns transformation is omitted in the final round.

Step 6: Ciphertext

After the final AddRoundKey operation, the resulting 128-bit State becomes the ciphertext.

AES Decryption

AES decryption reverses the encryption transformations. The inverse operations are:

Encryption	Decryption
SubBytes	InvSubBytes
ShiftRows	InvShiftRows
MixColumns	InvMixColumns
AddRoundKey	AddRoundKey

The same AddRoundKey operation is used because XOR is its own inverse.

The expanded key is used in reverse order during decryption. However, the normal AES decryption algorithm is not simply identical to the encryption algorithm because of the order of transformations. Each transformation is reversible, ensuring that the original plaintext can be recovered correctly from the ciphertext when the correct key is used.

Equivalent Inverse Cipher

The ordinary decryption algorithm has a different transformation order from encryption. This may require separate implementation structures in some situations.

An equivalent form of the decryption algorithm can be constructed so that it has a structure similar to encryption, except that inverse transformations are used and the key schedule is modified appropriately.

The encryption round is:

```
SubBytes
→ ShiftRows
→ MixColumns
→ AddRoundKey
```

The normal decryption round is:

```
InvShiftRows
→ InvSubBytes
→ AddRoundKey
→ InvMixColumns
```

Certain transformations can be rearranged because of their properties. For example, InvShiftRows changes byte positions, whereas InvSubBytes changes byte values independently.

Avalanche Effect-The **avalanche effect** means that a small change in plaintext or key should produce a large change in ciphertext.

Example

If one bit of plaintext changes:

Plaintext A → Ciphertext A

Plaintext B

(one-bit change)

↓

Ciphertext B

(many changes)

Importance

It makes it difficult for an attacker to identify relationships between:

- Plaintext
- Ciphertext
- Key

Important Characteristics of AES

AES has several important characteristics:

AES is a symmetric-key algorithm, meaning that the same secret key is fundamentally used for encryption and decryption.

AES is a block cipher and processes data in blocks of **128 bits**.

AES supports three key sizes: 128, 192, and 256 bits.

AES does not use a Feistel structure. It processes the complete State using substitution and permutation operations.

AES uses four major transformations: SubBytes, ShiftRows, MixColumns, and AddRoundKey.

AES provides confusion mainly through nonlinear byte substitution.

AES provides diffusion through ShiftRows and MixColumns.

AES uses GF(2^8) arithmetic, especially in the construction of transformations such as MixColumns.

AES uses expanded round keys, rather than repeatedly using the original key directly.

AES transformations are reversible, allowing correct decryption when the correct secret key is available.

AES Implementation

AES can be implemented in:

- Software
- Hardware
- Embedded systems
- General-purpose processors

Important Implementation Operations

- S-box lookup
- XOR operations
- Byte shifting
- GF(2^8) multiplication
- Key expansion

Security Consideration

A mathematically secure algorithm can still be vulnerable if its implementation leaks information through timing, power consumption, or other physical behavior.

Advantages of AES

Advantages

- Strong security
- Larger key sizes
- Efficient encryption
- Efficient hardware implementation
- Efficient software implementation
- Suitable for different applications
- Better security margin than DES
- Standardized by NIST

AES is widely used for protecting sensitive information.

FINITE FIELD ARITHMETIC

In **AES (Advanced Encryption Standard)**, all operations are performed on **8-bit bytes**. The important arithmetic operations are:

- Addition
- Multiplication
- Division

These operations are performed over a finite field called $GF(2^8)$. Since 2^8=256, the field contains **256 different elements**.

A **finite field** is a set containing a finite number of elements in which arithmetic operations such as:

- Addition
- Subtraction
- Multiplication
- Division (except division by zero)

can be performed and the result remains within the same set.

Example

The set:

$$Z_p = \{0, 1, 2, \ldots, p-1\}$$

where p is a prime number, is a finite field.

For example:

$$Z_5 = \{0, 1, 2, 3, 4\}$$

All arithmetic is performed modulo 5.

Example

$$4 + 3 = 7$$

But in Z_5:

$$7 \mod 5 = 2$$

Therefore:

$$\boxed{4 + 3 = 2 \text{ in } Z_5}$$

Galois Field GF(2^n)

A finite field containing 2^n elements is represented as:

$$\boxed{GF(2^n)}$$

In AES:

$$\boxed{GF(2^8)}$$

is used.

The elements are represented using **polynomials with binary coefficients.**

The coefficients can only be:

$$0 \text{ or } 1$$

Polynomial Representation

A general polynomial in $GF(2^n)$ is:

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1x + a_0$$

or:

$$\boxed{f(x) = \sum_{i=0}^{n-1} a_i x^i}$$

where:

$$\boxed{a_i = 0 \text{ or } 1}$$

Thus, every element of the field is represented using a binary polynomial.

Arithmetic Operations in Finite Fields:

Rule 1: Polynomial Arithmetic is Used.

Example

$$(x+1)(x^2+1)$$

Multiply each term:

$$= x(x^2+1) + 1(x^2+1)$$

$$= x^3 + x + x^2 + 1$$

Therefore:

$$\boxed{(x+1)(x^2+1) = x^3 + x^2 + x + 1}$$

However, if the degree becomes too large, it must be reduced.

Rule 2: Addition is Performed Modulo 2.

The coefficients are added modulo 2.

Addition	Result
$0+0$	0
$0+1$	1
$1+0$	1
$1+1$	0

Therefore:

$$\boxed{1+1=0}$$

This is exactly the same as the **XOR operation**.

Important Exam Point

$$\boxed{\text{Addition in } GF(2^n) = \text{XOR operation}}$$

Rule 3: Reduce Higher-Degree Polynomials

In GF(2^n), the degree of a polynomial must be less than n.

If multiplication produces a polynomial with degree greater than or equal to n, it must be reduced using an **irreducible polynomial**.

Suppose the irreducible polynomial is:

m(x)

Then:

$$\boxed{r(x) = f(x) \mod m(x)}$$

where $r(x)$ is the remainder.

AES Finite Field $GF(2^8)$

AES operates on:

$$\boxed{GF(2^8)}$$

Therefore, every element is represented using an 8-bit binary number.

The corresponding polynomial is:

$$a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

where:

$$a_i \in \{0, 1\}$$

There are:

$$\boxed{256 \text{ elements}}$$

in $GF(2^8)$.

AES Irreducible Polynomial

AES uses the irreducible polynomial:

$$\boxed{m(x) = x^8 + x^4 + x^3 + x + 1}$$

This polynomial is used to reduce the result when multiplication produces terms of degree 8 or greater.

Important Exam Point

$$\boxed{m(x) = x^8 + x^4 + x^3 + x + 1}$$

This is one of the most important formulas in AES finite field arithmetic.

Suppose polynomial multiplication produces:

$$f(x)$$

whose degree is greater than or equal to 8.

Then AES performs:

$$\boxed{f(x) \mod (x^8 + x^4 + x^3 + x + 1)}$$

The remainder becomes the final result.

This ensures that the result always remains within:

$$\boxed{GF(2^8)}$$

Relationship Between Binary and Polynomials

An 8-bit binary value can be represented as a polynomial.

Example

Consider:

$$01010011$$

The powers corresponding to the 1s are:

$$x^6, \quad x^4, \quad x, \quad 1$$

Therefore:

$$\boxed{01010011 = x^6 + x^4 + x + 1}$$

This polynomial representation makes AES arithmetic possible.

More on Symmetric Ciphers:

Multiple Encryption, Triple DES, Block Cipher Modes of Operation, Stream Cipher and RC4.

MULTIPLE ENCRYPTION AND TRIPLE DES (3DES):

DES uses a **56-bit key**. With the development of powerful computers, DES became vulnerable to a **brute-force attack**, because an attacker can theoretically try all possible keys. Therefore, one approach to increase the security of DES is **Multiple Encryption**. Which means, **Apply the DES algorithm more than once to the same plaintext using multiple keys.**

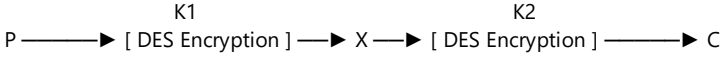
Main Types

1. Double DES / Double Encryption
2. Triple DES / Triple Encryption / 3DES

DOUBLE DES-Double DES uses **two stages of encryption** and **two different keys**. Let, Plaintext = P, First key = K_1, Second key = K_2, Ciphertext = C.

Encryption: $C=E(K_2,E(K_1,P))$

Diagram



Explanation:

1. Plaintext P is encrypted using key K_1.
2. The intermediate result is called X.
3. X is encrypted again using key K_2.
4. The final output is ciphertext C.

Thus: $X=E(K_1,P)$

$C=E(K_2,X)$

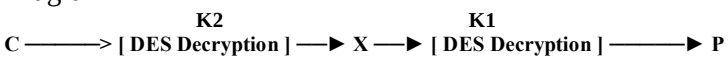
Therefore:

$$C = E(K_2, E(K_1, P))$$

Double DES Decryption: For decryption, operations are performed in reverse order.

$P=D(K_1,D(K_2,C))$

Diagram



Suppose for every pair of keys K1 and K2, we could find a key K3 such that: $E(K_2, E(K_1, P)) = E(K_3, P)$

If this were true, then double encryption would provide no additional security. Because,

Double DES , P → Encrypt with K1 → Encrypt with K2 → C
would be equivalent to Single DES, P → Encrypt with K3 → C.In that case, using two encryptions would be useless.

DES maps a 64-bit plaintext block into a 64-bit ciphertext block. For a particular key, DES performs a particular permutation or mapping. The number of possible mappings of 64-bit blocks is extremely large (2^64)!. DES has only 2^56 possible keys. Therefore, it is reasonable to conclude that performing DES twice with two different keys can produce mappings that cannot be produced by a single DES operation. So, Double DES is not generally equivalent to single DES. However, Double DES still has a serious weakness called the **Meet-in-the-Middle Attack**.

MEET-IN-THE-MIDDLE ATTACK- Although Double DES uses two 56-bit keys, it is vulnerable to the **Meet-in-the-Middle Attack**.

Suppose: C=E(K2,E(K1,P))

Let the intermediate value be **X**.

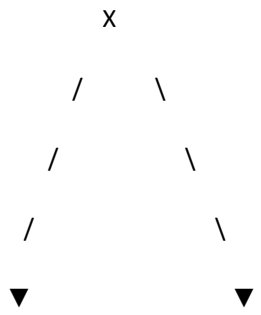
Then: X=E(K1,P)

Also: X=D(K2,C)

Therefore:

$\boxed{E(K_1, P) = D(K_2, C)}$ This is the basic idea of the Meet-in-the-Middle Attack.

Meet-in-the-Middle Attack Diagram:



P —> Encrypt with K1 Decrypt with K2 ◀— C

E(K1,P) D(K2,C)

The attacker calculates values from both directions and looks for a common intermediate value **X**. That is why the attack is called **MEET-IN-THE-MIDDLE ATTACK.**

STEPS OF MEET-IN-THE-MIDDLE ATTACK:

Assume that the attacker knows (P, C) , where: P = Known plaintext ,C = Corresponding ciphertext .

Step 1: Encrypt from the Plaintext Side

For every possible value of K1: X=E(K1,P) .Since DES has a 56-bit key: 2^{56} ,possible values of K1 are tried. The attacker stores all intermediate values in a table.

K1 X = E(K1,P)

K1(1) ————————> X1

K1(2) ——————> X2

K1(3) ——————> X3

...

...

K1(n) ——————> Xn, The table is sorted according to the value of **X**.

Step 2: Decrypt from the Ciphertext Side

For every possible value of K2: $X=D(K2,C)$.

Again, 2^{56}

Possible keys are tried.

K2 X = D(K2,C)

K2(1) ——————> X1

K2(2) ——————> X2

K2(3) ——————> X3

...

...

K2(n) ——————> Xn

Step 3: Look for a Match

The attacker checks whether: E(K1,P)=D(K2,C).

If a match is found:

Encryption side Decryption side

E(K1,P) = X = D(K2,C).

then the corresponding K1 and K2 are candidate keys.

Step 4: Verify the Keys

The candidate keys are tested using another known plaintext-ciphertext pair.

If: C=E(K2,E(K1,P))

is correct for another pair, the keys are accepted.

Complexity of Meet-in-the-Middle Attack: A brute-force attack on Double DES would apparently require: 2^{112} operations because there are two 56-bit keys .However, the Meet-in-the-Middle Attack requires approximately: 2^{56} encryptions and:2^{56} decryptions, along with a large amount of storage. Thus, Double DES does not provide the expected 112-bit security. Double DES is vulnerable to the Meet-in-the-Middle Attack. Therefore , simply applying DES twice does not provide 112-bit security.

TRIPLE DES (3DES) -To overcome the weakness of Double DES, DES can be applied **three times**.

This is called:

- Triple DES
- 3DES
- Triple Data Encryption Algorithm (TDEA)

There are two important versions:

1. Two-Key Triple DES
2. Three-Key Triple DES

MULTIPLE ENCRYPTION DIAGRAM- The diagram in the source represents Double Encryption and Triple Encryption.

(a) Double Encryption:

K1 **K2**

P ——► Encrypt ——► Encrypt ——► C

Mathematically: C=E(K2,E(K1,P))

Decryption: P=D(K1,D(K2,C))

(b) Triple Encryption:

Triple DES can use either:

- Two keys: K1,K2,K1
- Three keys: K1,K2,K3
- General representation:

K1 K2 K3

P ——► DES ——► DES ——► DES ——► C

TWO-KEY TRIPLE DES-Two-key Triple DES uses the Encrypt–Decrypt–Encrypt (EDE) **sequence.**

The keys are:

- First stage = K1
- Second stage = K2
- Third stage = K1

Therefore, the first and third keys are the same.

Encryption Formula: C=E(K1,D(K2,E(K1,P))).

Step-by-Step,

TWO-KEY TRIPLE DES

K1 K2 K1

P ——> Encrypt ——▶A ——▶Decrypt ——▶B ——▶Encrypt ——▶C.

Where:

A=E(K1,P) B=D(K2,A) C=E(K1,B)

Therefore: C=E(K1,D(K2,E(K1,P)))

TWO-KEY 3DES DECRYPTION -The operations are reversed.

P=D(K1,E(K2,D(K1,C)))

Diagram

K1 K2 K1

C ——> Decrypt ——▶ B ——▶ Encrypt ——▶ A ——▶ Decrypt ——▶ P

EDE Sequence in Triple DES: Triple DES use Encrypt–Decrypt–Encrypt instead of Encrypt–Encrypt–Encrypt, The main reason is **backward compatibility with DES**.

Suppose: K1=K2

Then: C=E(K1,D(K1,E(K1,P)))

Since: D(K1,E(K1,P))=P

we get: C=E(K1,P)C=E(K_1,P)

Thus, Triple DES behaves exactly like ordinary DES.

C=E(K1,D(K1,E(K1,P)))

Since encryption followed by decryption using the same key cancels: D(K1,E(K1,P))=P

Therefore: C=E(K1,P)

Similarly: P=D(K1,C)

EDE allows Triple DES systems to remain compatible with older DES systems.

SECURITY OF TWO-KEY TRIPLE DES : Two-key 3DES uses two independent 56-bit keys. Therefore, the total key length is: 56+56=112 bits

It is much stronger than:

- Single DES

- Double DES

However, several theoretical attacks have been proposed.

1. Chosen-plaintext attack
2. Known-plaintext attack

Although these attacks are not considered practical in normal situations, they are important from a cryptographic analysis point of view.

KNOWN-PLAINTEXT ATTACK ON TWO-KEY 3DES- The two-key 3DES encryption is:

C=E(K1,D(K2,E(K1,P)))

Let: A=E(K1,P) and: B=D(K2,A)

Therefore:

P ——► E(K1) —► A —► D(K2) —► B —► E(K1) —► C

Diagram: Known-Plaintext Attack Concept

K1 K2 K1

P ————————► [Encrypt] —► A —► [Decrypt] —► B —► [Encrypt] ———► C.

The attacker knows several :(P,C) pairs. The intermediate value A is unknown. The attacker selects a possible value: **A=a** , and attempts to find keys that satisfy the encryption relationship.

STEPS OF THE KNOWN-PLAINTEXT ATTACK

Step 1: Obtain Known Plaintext-Ciphertext Pairs

The attacker obtains: (P,C) pairs. These pairs are placed in a table and sorted according to plaintext.

Table 1

Plaintext (P)	Ciphertext (C)
P1	C1
P2	C2
P3	C3
P4	C4
...	

Step 2: Select an Arbitrary Value of A

Choose an arbitrary value a , as a possible value of the intermediate variable A.

For every possible key: K1=i

calculate: Pi=D(i,a)

If this plaintext matches an entry in Table 1, then calculate: B=D(i, C)

and store:

- Candidate key i
- Corresponding value of B.

Table 2

Candidate K1	Intermediate B
i1	B1
i2	B2
i3	B3
...	

The table is sorted according to **B.**

Step 3: Search for K_2

For every possible key: K2=j

Calculate : Bj=D(j,a)

Now compare Bj with values in Table 2.

If a match is found: Bj=B

then the corresponding pair: (K1,K2) becomes a candidate pair of keys.

Step 4: Test Candidate Keys

Each candidate pair is tested using additional plaintext-ciphertext pairs.If the candidate keys produce the correct ciphertext, they are accepted. If no candidate succeeds, the process is repeated with another value of: a

THREE-KEY TRIPLE DES- Three-key Triple DES uses three independent keys: **K1,K2,K3.**

The EDE sequence is:

K1 K2 K3

P ——————► Encrypt ——————► Decrypt ——————► Encrypt ——————► C.

Encryption Formula: **C=E(K3,D(K2,E(K1,P)))**

Step-by-Step

First: A=E(K1,P)

Second: B=D(K2,A)

Third: C=E(K3,B)

Therefore: C=E(K3,D(K2,E(K1,P)))

THREE-KEY 3DES DECRYPTION:

Decryption is performed in reverse order: P=D(K1,E(K2,D(K3,C)))

Diagram K3 K2 K1

C ——————► Decrypt ——————► Encrypt ——————► Decrypt ——————► P

KEY SIZE OF THREE-KEY 3DES:

Each DES key has: 56 bits.

Since three keys are used: 56×3=168 bits

Therefore, the actual key material consists of: 168 bits.

However, Triple DES does not provide the full equivalent security of a simple 168-bit ideal cipher. Cryptanalytic attacks reduce its effective security level significantly compared with the naive 168-bit expectation.

BACKWARD COMPATIBILITY OF THREE-KEY 3DES

Backward compatibility can be achieved by making keys equal.

For example: K1=K2=K3 ,Then the Triple DES operation effectively reduces to DES. Also, configurations such as making appropriate keys equal can provide compatibility with two-key Triple DES.

Thus, Triple DES provides a transition path between:

Single DES

↓

Two-Key Triple DES

↓

Three-Key Triple DES

COMPARISON: DES, DOUBLE DES AND TRIPLE DES

Feature	DES	Double DES	Two-Key 3DES	Three-Key 3DES
Number of stages	1	2	3	3
Number of keys	1	2	2	3
Key size	56 bits	112 bits	112 bits	168 bits
Main sequence	E	E-E	E-D-E	E-D-E
Security	Weak today	Vulnerable to MITM	Stronger	Stronger than two-key version
DES compatibility	Yes	No special feature	Yes	Can provide compatibility

TWO-KEY VS THREE-KEY TRIPLE DES

Two-Key 3DES	Three-Key 3DES
Uses K1,K2,	Uses K1,K2,K3
Two independent keys	Three independent keys
112 bits of key material	168 bits of key material
Less secure than 3-key version	More secure than 2-key version
EDE sequence	EDE sequence
Historically widely used	Preferred over two-key version in stronger deployments

ADVANTAGES OF TRIPLE DES

1. Provides stronger security than DES.
2. Uses the already established DES algorithm.
3. Provides backward compatibility with DES through the EDE structure.
4. Is stronger against brute-force attacks than single DES.
5. Has been widely used in security applications and standards.

DISADVANTAGES OF TRIPLE DES

1. It is slower than single DES because DES operations are performed three times.
2. It has a smaller block size than modern algorithms such as AES.
3. It is less efficient for modern high-speed applications.
4. It has largely been replaced by AES in many modern applications.
5. Two-key 3DES provides less security than the simple key-length calculation may suggest.

INTRODUCTION TO BLOCK CIPHER MODES OF OPERATION

A block cipher encrypts data in blocks of fixed size.

For example:

- DES operates on blocks of 64 bits.
- AES operates on blocks of 128 bits.

Suppose we have a long message: P=P1P2P3…PN , where: P1,P2,P3,…,PN. are individual plaintext blocks.

A **block cipher mode of operation** specifies how multiple plaintext blocks are processed using a block cipher algorithm.

The important modes are:

1. Electronic Codebook (ECB)
2. Cipher Block Chaining (CBC)
3. Cipher Feedback (CFB)
4. Output Feedback (OFB)
5. Counter (CTR)

Suppose we simply divide a message into blocks: This simple method has security limitations.

Plaintext Message

P1 P2 P3 P4

| | | |

▼ ▼ ▼ ▼

DES DES DES DES

| | | |

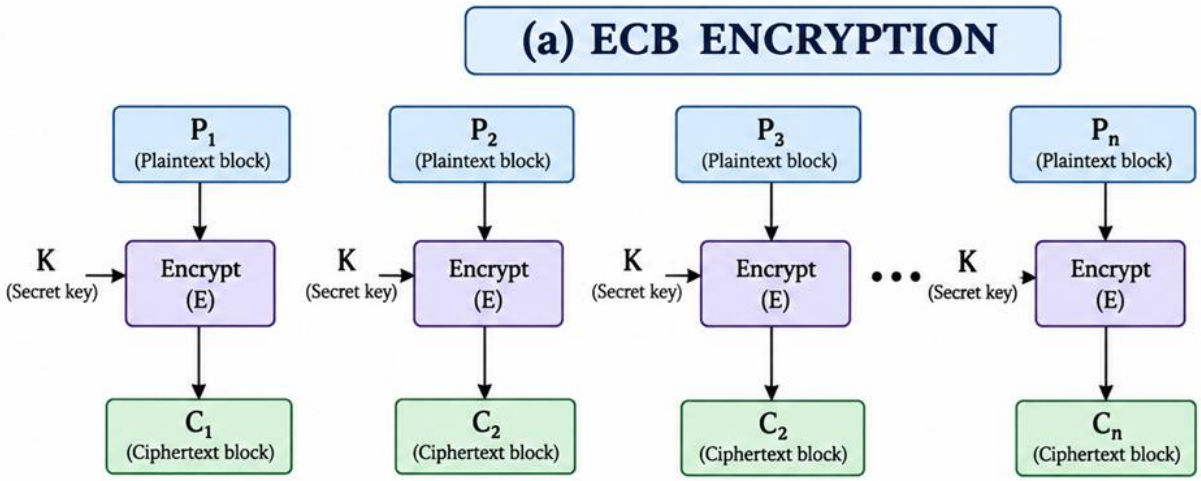
▼ ▼ ▼ ▼

C1 C2 C3 C4

For example, if: P1=P2

then the corresponding ciphertext may also be identical, C1=C2. Thus, patterns in the original message may become visible. Different modes of operation solve this problem in different ways.

ELECTRONIC CODEBOOK (ECB) MODE-ECB is the simplest mode of operation. The plaintext is divided into blocks, and each block is encrypted independently using the same secret key.



There is **no connection** between:

- Block 1 and Block 2

- Block 2 and Block 3

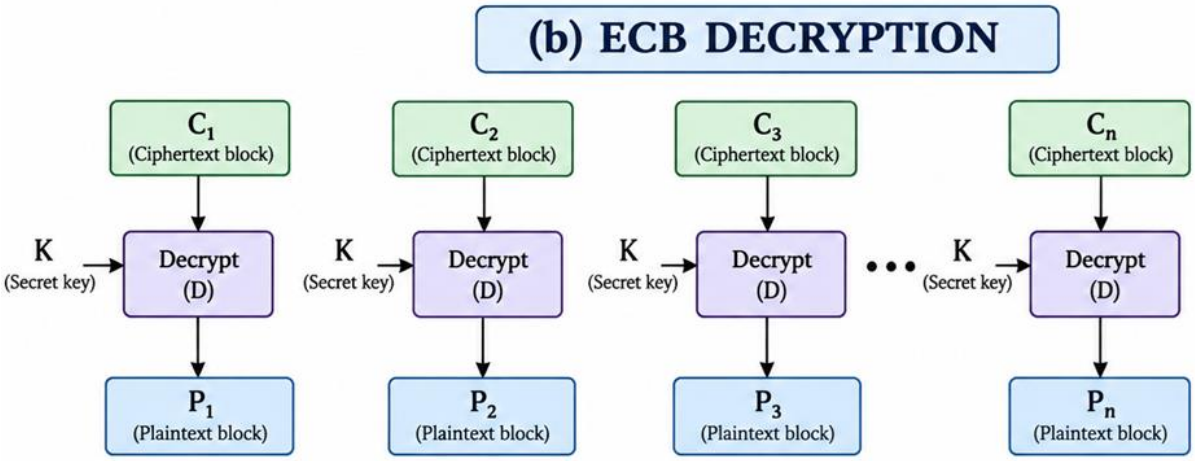
Each block is treated independently.

ECB Encryption: Cj=E(K,Pj)

For every plaintext block: **Cj=E(K,Pj)**

Where: Pj= jth plaintext block , Cj = jth ciphertext block ,K = secret key ,E = encryption algorithm

ECB DECRYPTION: Pj=D(K,Cj)



IMPORTANT CHARACTERISTIC OF ECB

Suppose: P1=P2.

Since the same key is used: E(K,P1)=E(K,P2)

Therefore: C1=C2.

This is the major weakness of ECB.

Consider a message containing repeated blocks ,Plaintext :A A A B B A.

After ECB encryption, Ciphertext : X X X Y Y X

An attacker may not know the actual values of A and X, but can observe: "These ciphertext blocks are repeating." Therefore, information about the structure of the plaintext is leaked. That's the reason ECB is not secure for Long Messages. Thus, ECB may reveal:

- Repeated data
- Message structure
- Repetitive fields
- Possible plaintext patterns

The message may not always contain a number of bits exactly divisible by the block size. For example, if DES has a block size of 64 bits and the last plaintext contains only 50 bits then additional bits are added. This process is called **Padding** .The last block is padded to create a complete block.

ADVANTAGES OF ECB

1. **Simple**: The simplest block cipher mode.

2. **Parallel Processing**: All blocks can be encrypted simultaneously.

P1 → Encrypt → C1

P2 → Encrypt → C2

P3 → Encrypt → C3

3. **Error Does Not Spread** : If an error occurs in C2, normally only P2 is affected. Other blocks can still be decrypted.

DISADVANTAGES OF ECB

1. Reveals patterns-Repeated plaintext blocks produce repeated ciphertext blocks.

2. Not suitable for highly structured messages-An attacker may identify repeated structures.

3. Blocks can be manipulated independently-Because blocks are independent, an attacker may attempt to rearrange ciphertext blocks.

EVALUATION CRITERIA FOR BLOCK CIPHER MODES:

1. OVERHEAD- means additional processing required compared with ECB.

For ECB: Plaintext → Encrypt → Ciphertext.

For CBC: Plaintext

↓

XOR with previous ciphertext

↓

Encrypt

↓

Ciphertext

CBC therefore requires additional operations. Thus, Overhead is the additional computational work required by a mode. **2. 2.ERROR RECOVERY -**Suppose an error occurs in ciphertext block Ci, A good mode may ensure that the error affects only a limited number of plaintext blocks. After some blocks, the system may recover. This property is called **Error Recovery.**
3. ERROR PROPAGATION- Suppose a bit error occurs during transmission. If an error in Ci , causes errors in Pi,Pi+1,Pi+2,…, then the error is propagating. Thus, Error propagation means that an error in one ciphertext block affects subsequent plaintext blocks.
4. DIFFUSION-Diffusion means spreading the influence of plaintext over ciphertext. The aim is to prevent a direct relationship between: Plaintext Pattern

↓

Ciphertext Pattern

A good mode should make the relationship between plaintext blocks and ciphertext blocks difficult to predict.
5. SECURITY-The mode should prevent an attacker from obtaining useful information about:

- Plaintext structure
- Repeated plaintext blocks
- Relationships between block

CIPHER BLOCK CHAINING (CBC) MODE

CBC was developed to overcome the main weakness of ECB. In ECB, Each block is encrypted independently. CBC provides a technique in which the input to the encryption algorithm is the **XOR of the plaintext block and the preceding ciphertext block.**Thus, the processing of the plaintext blocks is **chained together**.

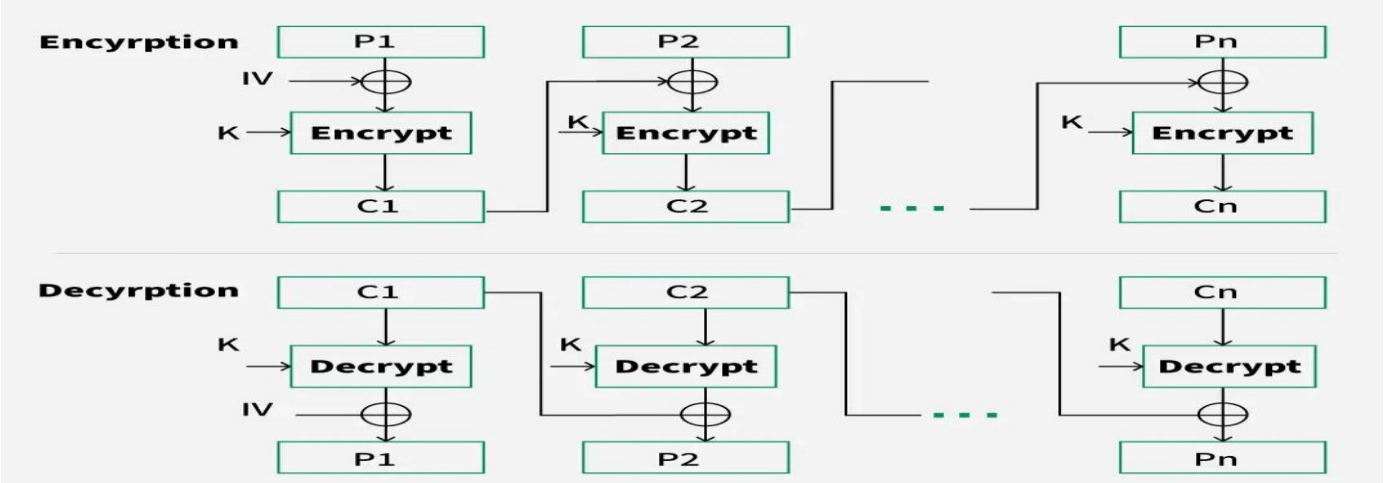
In CBC, the input to the encryption function for each plaintext block consists of the plaintext block, and the preceding ciphertext block. Therefore, the encryption input is not simply a fixed transformation of the plaintext block. This means that even if the same plaintext block is repeated, it does not necessarily produce the same ciphertext block. This chaining prevents the patterns in the plaintext from being exposed. The CBC mode also requires that the last block be **padded to a full b bits if it is a partial block.**

CBC Encryption:

Let:

- Pi = plaintext block
- Ci = ciphertext block
- K = encryption key
- IV = Initialization Vector
- E = encryption algorithm

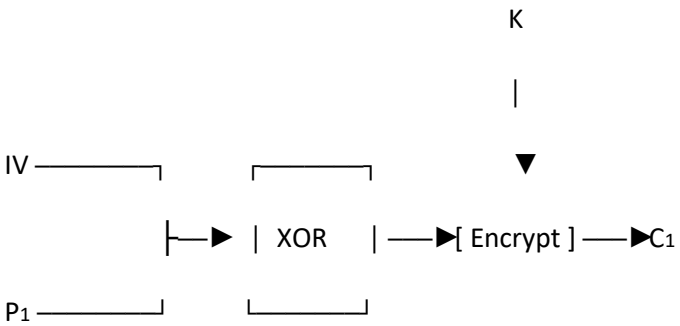
For the first plaintext block, an **Initialization Vector (IV)** is XORed with the first block of plaintext. The result is then encrypted. Therefore, $C1=E(K,[P1 \oplus IV])$.For subsequent blocks, the preceding ciphertext block is XORed with the current plaintext block. Thus, $Cj=E(K,[Cj-1 \oplus Pj])$ Where, j=2,…,N



Here we can observe that, C1 is used with P2.and C2 is used with P3 and so on. This forms the **cipher block chain.**

CBC ENCRYPTION – FIRST BLOCK

For the first block, there is no previous ciphertext. Therefore, an **Initialization Vector (IV)** is used.

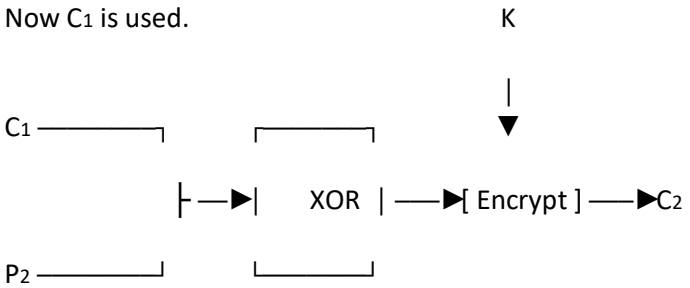


First: $IV \oplus P_1$

Then: $C_1=E(K, IV \oplus P_1)$

CBC ENCRYPTION – SECOND BLOCK

Now C_1 is used.



Therefore: $C_2=E(K, C_1 \oplus P_2)$

GENERAL CBC FORMULA

For the first block: $C_1=E(K, IV \oplus P_1)$

For all subsequent blocks: **Cj=E(K,Cj−1⊕Pj), j=2,…….N**

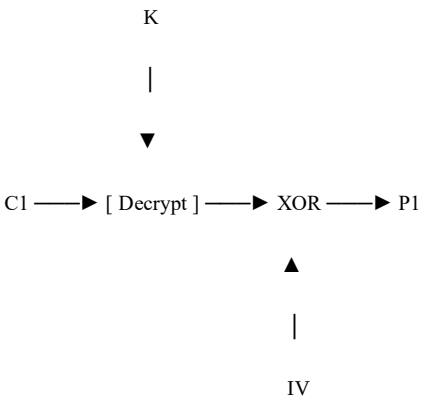
Observe the relationship:

P1 + IV
↓
C1
↓
P2 + C1
↓
C2
↓
P3 + C2
↓
C3

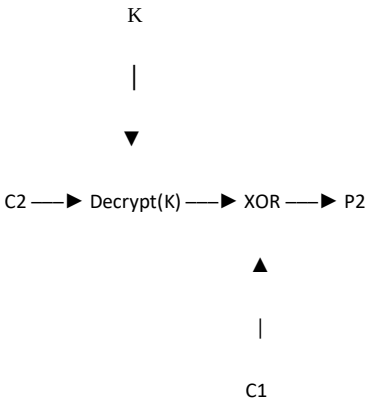
Each ciphertext block influences the encryption of the next plaintext block. Therefore, the blocks form a Ciphertext Chain.
CBC DECRYPTION-For decryption, each ciphertext block is first passed through the decryption algorithm.The result is then XORed with the preceding ciphertext block to produce the plaintext block.For the first block, the IV is used instead of a preceding ciphertext block.

For encryption: Cj=E(K,Cj−1⊕Pj)

To decrypt for the first block: P1=D(K,C1)⊕IV



SECOND BLOCK:



Therefore:

$P_2=D(K,C_2)\oplus C_1$

GENERAL CBC DECRYPTION:

$Pj=D(K,Cj)\oplus Cj-1$, j=2,……N

Encryption: $Cj=E(K,Cj-1\oplus Pj)$

Apply decryption algorithm: $D(K,Cj)=D(K,E(K,[Cj-1\oplus Pj]))$

Therefore: $D(K,Cj)=Cj-1\oplus Pj$

XORing both sides with Cj−1:

$Cj-1\oplus D(K,Cj)=Cj-1\oplus Cj-1\oplus Pj$

Since: $Cj-1\oplus Cj-1=0$

Therefore: $0\oplus Pj=Pj$

Hence: $Pj=D(K,Cj)\oplus Cj-1$

INITIALIZATION VECTOR (IV)

For the first block of ciphertext ,$C_1=E(K,[P_1\oplus IV])$

During decryption: $P_1=D(K,C_1)\oplus IV$

The IV is a data block that is the same size as the cipher block. Therefore, if the cipher has a block size of b bits, the IV is also b bits.

Encryption:

$C_1=E(K,[P_1\oplus IV])$

$Cj=E(K,[Cj-1\oplus Pj])$, j=2,…,N

Decryption:

$P_1=D(K,C_1)\oplus IV$

$Pj=D(K,Cj)\oplus Cj-1$, j=2,…,N.

These are the main CBC equations

REQUIREMENTS OF IV

The IV must:

1. Be known to sender and receiver-Both parties require the IV.

2. Be unpredictable-An attacker should not be able to predict the IV associated with a future plaintext.

3. Be protected from unauthorized changes-An attacker should not be able to modify the IV**.**

If an opponent can cause the receiver to use a different value for the IV, the opponent can cause selected bits in the first plaintext block to be inverted. The first-block equations are: $C_1=E(K,[IV \oplus P_1])$

And $P_1=IV \oplus D(K,C_1)$ then considers a change in selected bits of the IV.

Using X[i] to denote the i-th bit of a b-bit quantity, it shows that changing a bit in the IV results in the corresponding bit of the received plaintext being changed. Therefore, If an opponent can predictably change bits in the IV, the corresponding bits of the received value of P1 can be changed.

Suppose an attacker changes a particular bit of the IV. Then the corresponding bit of P1can also be changed. Therefore, **Changing a bit in the IV can change the corresponding bit in the first recovered plaintext block.** This is an important security point.

So long as it is unpredictable, the specific choice of IV is unimportant. It then refers to **NIST SP 800-38A**, which recommends two possible methods.

First method, Apply the encryption function, under the same key used for encryption of the plaintext, to a nonce.The nonce must be a data block that is unique to each execution of the encryption operation. For Example, the nonce may be a counter, a timestamp, or a message number.

Second method, is to generate a random data block using a random generator.

Nonce- A time-varying value that has at most a negligible chance of repeating.

Examples include:

- a random value generated a new for each use
- a timestamp
- a sequence number
- some combination of these

The purpose is to provide a value that changes with each execution.

Padding-The last block must be padded to a full b bits if it is a partial block.

CBC ADVANTAGES

1. Repeated plaintext blocks do not directly produce repeated ciphertext blocks.
2. Plaintext patterns are hidden.
3. More secure than ECB for long messages.
4. Useful for confidentiality.
5. Can also be used in authentication applications.

CBC DISADVANTAGES

1. Encryption is sequential.
2. Cannot easily encrypt all blocks in parallel.

3. Padding is required for an incomplete last block.
4. Requires an IV.
5. Errors can affect more than one plaintext block.

CIPHER FEEDBACK MODE (CFB)

For **AES, DES, or any block cipher**, encryption is performed on a block of b bits.

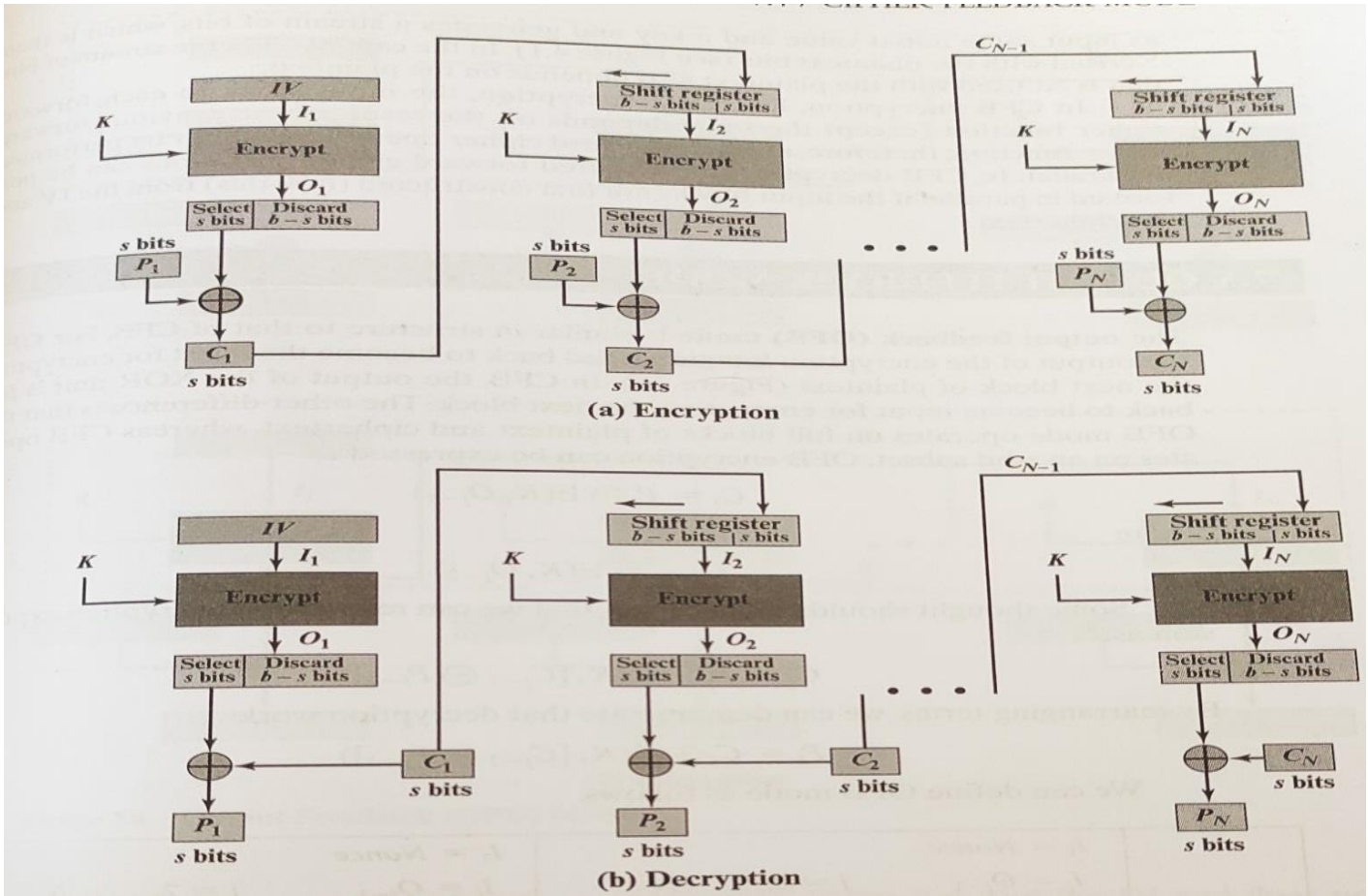
In the case of:

- DES, b=64
- AES, b=128

However, it is possible to convert a block cipher into a stream cipher by using one of the following modes:

1. Cipher Feedback (CFB) mode
2. Output Feedback (OFB) mode
3. Counter (CTR) mode

Stream Cipher-A stream cipher eliminates the need to pad a message so that it becomes an integral number of blocks. It can also operate in **real time**. For example, if a character stream is being transmitted, each character can be encrypted and transmitted immediately using a character-oriented stream cipher. One desirable property of a stream cipher is that, The ciphertext should be the same length as the plaintext. For example, if **8-bit characters** are being transmitted, each character should be encrypted to produce an **8-bit ciphertext output**. If more than 8 bits are produced, transmission capacity is wasted.



The image shows the CFB scheme using $\mathbf{I_1}$. It assumes that the unit of transmission is s bits. A common value is s=8. Thus, instead of processing complete blocks of b bits, the plaintext is divided into segments of s bits. For example: P1,P2,P3,…,PN where each plaintext unit contains s bits. As with CBC, the units of plaintext are **chained together**. The ciphertext of any plaintext unit is a function of **all the preceding plaintext**. This is achieved through the **shift register**. The shift register is used as the input to the encryption function.

CFB Encryption- Initially, the input to the encryption function is a **b-bit shift register**. This register is initially set to the **Initialization Vector (IV)**.

Thus: $I_1=IV$

The key K is also supplied to the encryption function.

The encryption function produces: $O_1=E(K,I_1)$

The leftmost (most significant) s bits of the output are selected. The remaining: $\mathbf{b-s}$ bits are discarded. Therefore, we select $MSBs(O_1)$

First Plaintext Segment: The selected s bits of the encryption output are XORed with the first plaintext segment P_1.

Therefore, $C_1=P_1 \oplus MSBs[E(K,IV)]$ This produces the first ciphertext segment C_1.The ciphertext segment C_1 is then transmitted.

Shift Register Operation

After producing C_1:

- The contents of the shift register are shifted **left by** s **bits**.
- C_1 is placed in the **rightmost (least significant)** s **bits** of the shift register.

The new contents of the shift register are then used as the input for the next encryption operation. This process continues until all plaintext units have been encrypted.

Second CFB Step- After, C_1 has been placed into the shift register, the register contents become I_2. The encryption function is applied,

$O_2=E(K,I_2)$

Again:

- the most significant s bits are selected;
- the remaining b−s bits are discarded.

The selected bits are XORed with P_2 to produce C_2. Thus, $C_2=P_2 \oplus MSBs(O_2)$

Continuing the Process -The same process is repeated for all plaintext segments.

The sequence is: $P_1 \rightarrow C_1$

Then $P_2 \rightarrow C_2$

Then $P_3 \rightarrow C_3$

and so on. Each newly generated ciphertext segment is inserted into the shift register and becomes part of the input for the next encryption operation.

For decryption, the **same scheme** is used. However, the received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit. It is the **encryption function** that is used, **not the decryption function**. **Thus, CFB decryption does not use the block cipher's decryption function.**

First CFB Decryption Step

The shift register is initially set to IV

Therefore, $I_1=IV$

The encryption function is applied $O_1=E(K,I_1)$

The most significant s bits are selected.

The received ciphertext C_1 is XORed with these selected bits.

Therefore, $P_1=C_1 \oplus MSBs[O_1]$

Since: $O_1=E(K,IV)$

we have: $P_1=C_1 \oplus MSBs[E(K,IV)]$

Why Decryption Works

$C_1=P_1 \oplus MSBs[E(K,IV)]$

Rearranging the terms gives:

$P_1=C_1 \oplus MSBs[E(K,IV)]$

This is the decryption equation.

The same reasoning applies to the **subsequent steps** in the process.

CFB Decryption Process

After C_1 is received:

1. The encryption function operates on the current shift-register contents.
2. The most significant s bits of the output are selected.
3. These bits are XORed with C_1.
4. This produces P_1.
5. The shift register is shifted left by s bits.
6. C_1 is placed in the rightmost s bits.
7. The new shift-register contents are used for the next operation.

The same process continues for: $C_2, C_3, \ldots, C_N$

Encryption

Initial register $\mathbf{I_1=IV}$

Subsequent register values

$I_j=LSB_{b-s}(I_{j-1}) \| C_{j-1}$

For $j=2,\ldots,N$

Here ‖ represents concatenation.

The shift register is shifted left by s bits, and C_{j-1} is placed into the rightmost s bits.

Encryption Output

For each step:

$O_j=E(K,I_j)$

for: $j=1,\ldots,N$

The most significant s bits are then selected.

Ciphertext Equation

$C_j=P_j \oplus MSB_s(O_j)$

for: $j=1,\ldots,N$

Thus, the complete CFB encryption process is:

$I_1=IV$
$I_j=LSB_{b-s}(I_{j-1}) \| C_{j-1}$
$O_j=E(K,I_j)$
$C_j=P_j \oplus MSB_s(O_j)$

CFB Decryption Equations

For decryption: $I_1=IV$

And $I_j=LSB_{b-s}(I_{j-1}) \| C_{j-1}$

for: $j=2,\ldots,N$

The encryption function is used: $O_j=E(K,I_j)$

for: $j=1,\ldots,N$

The plaintext is recovered by: $P_j=C_j \oplus MSB_s(O_j)$ for: $j=1,\ldots,N$

Complete CFB Definition

Encryption

$I_1=IV$
$Ij=LSB_{b-s}(Ij-1)\|Cj-1$
$Oj=E(K,Ij)$
$Cj=Pj\oplus MSB_s(Oj)$

Decryption

$I_1=IV$
$Ij=LSB_{b-s}(Ij-1)\|Cj-1$
$Oj=E(K,Ij)$
$Pj=Cj\oplus MSB_s(Oj)$

Meaning of MSBs
MSBs(X) as the **most significant s bits of X**. Therefore MSBs(Oj) means that only the leftmost s bits of the encryption output Oj are used for the XOR operation. The remaining b−s bits are discarded.

The shift register contains b bits.

Initially: $I_1=IV$

After each ciphertext segment is produced:

- the contents are shifted left by s bits;
- the new ciphertext segment is placed into the rightmost s bits.

Therefore: $Ij=LSB_{b-s}(Ij-1)\|Cj-1$

This is the feedback mechanism in CFB.

Although CFB can be viewed as a stream cipher, it does not confirm to the typical construction of a stream cipher.

In a typical stream cipher:

- the cipher takes an initial value and a key;
- it generates a stream of bits;
- this stream is XORed with the plaintext bits.

In CFB, the stream of bits that is XORed with the plaintext **also depends on the plaintext**.This is because the ciphertext is fed back into the shift register.

Difference From Typical Stream Cipher Construction

In a typical stream cipher, the generated stream depends on:

- the initial value
- the key

But in CFB, the generated stream, after the first step, depends on the previous ciphertext.

Therefore, The stream of bits used for XORing with the plaintext also depends on the plaintext. This is an important characteristic of CFB.

In CFB encryption, like CBC encryption, the input block to each forward cipher function, except the first, depends on the result of the previous cipher function. Therefore, Multiple forward cipher operations cannot be performed in parallel. Each operation depends on the result of the preceding operation.

In CFB decryption, the required forward cipher operations **can be performed in parallel if the input blocks are first constructed in series from the ciphertext**. The input blocks to the encryption function depend on the ciphertext. Once these input blocks have been constructed, the forward cipher operations can be performed.

Output Feedback Mode

The **Output Feedback (OFB) mode** is similar in structure to **Cipher Feedback (CFB) mode**. The main difference is in what is **fed back**. **In OFB,** The **output of the encryption function** is fed back to become the input for encrypting the next block of plaintext. **In CFB,** The **output of the XOR unit** is fed back to become the input for encrypting the next block. Therefore, the important difference is:

OFB → encryption output is fed back

CFB → ciphertext output is fed back

Difference in the Unit of Operation-Another difference between OFB and CFB is OFB operates on full blocks of plaintext and ciphertext. CFB operates on an s-bit subset. Thus, OFB operates on the complete block.

OFB Encryption Equation
OFB encryption as: $Cj=Pj \oplus E(K,Oj-1)$ where: $Oj-1=E(K,Oj-2)$ .The encryption output is therefore repeatedly used as the input to the next encryption operation.

First OFB Operation

The first input is an initial value called the **IV**. The initial value is represented as a **Nonce**:

$I_1=Nonce$

The encryption function is applied to this value: $O_1=E(K,I_1)$

The resulting output $O_1$is then XORed with the first plaintext block.

Therefore $C_1=P_1 \oplus O_1$

Second OFB Operation

The output from the first encryption operation is used as the input to the second encryption operation.

Thus: $I_2=O_1$

Then: $O_2=E(K,I_2)$

The second plaintext block is XORed with O_2:

$C_2=P_2 \oplus O_2$

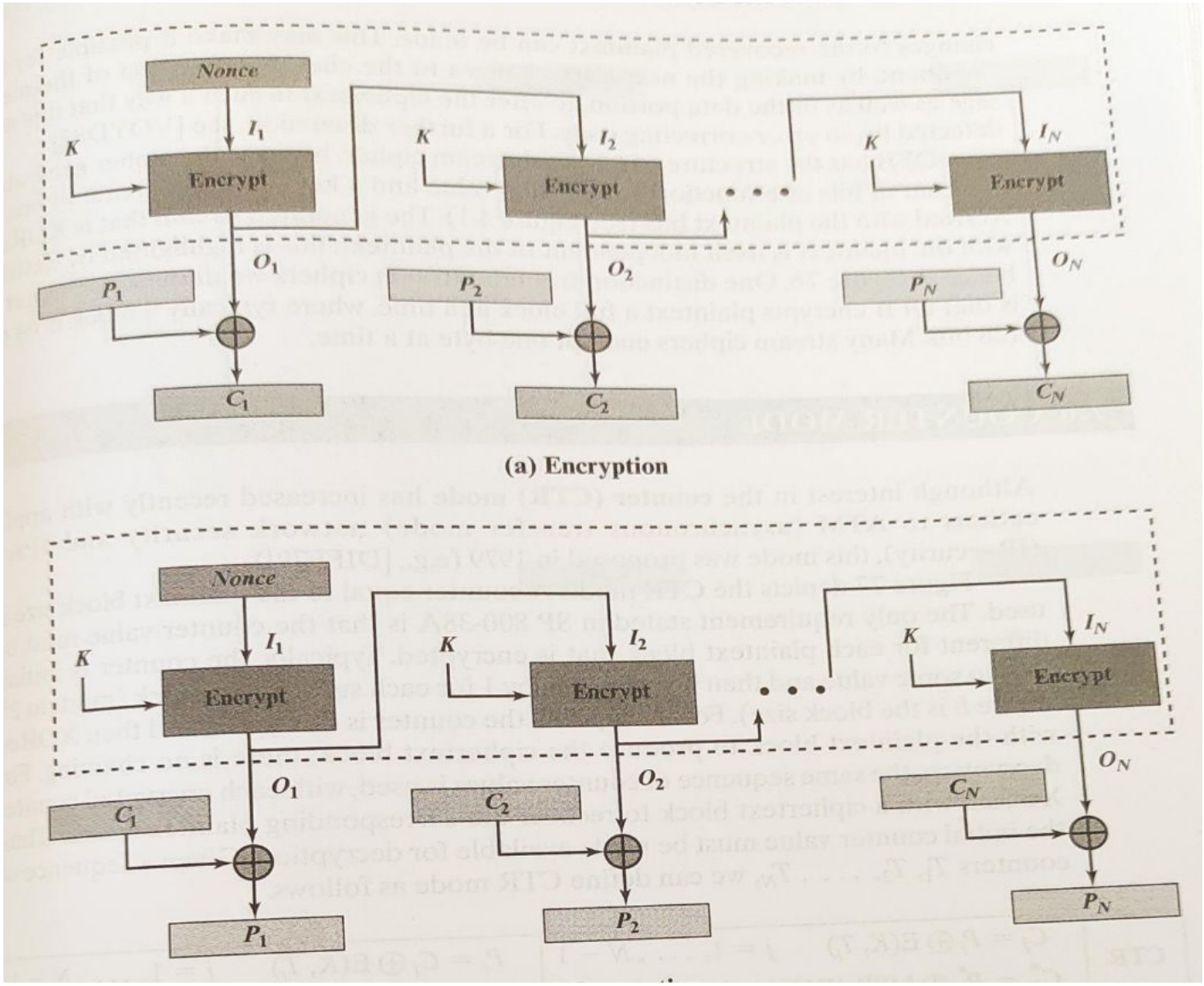
Third OFB Operation

The output O_2 becomes the input to the next encryption operation: $I_3=O_2$

Then: $O_3=E(K,I_3)$

The third plaintext block is XORed with O_3: $C_3=P_3 \oplus O_3$

The same process continues for all plaintext blocks.



Rewriting the Encryption Expression Some thought should convince you that we can rewrite the encryption expression as: $Cj=Pj \oplus E(K,[Cj-1 \oplus Pj-1])$

This follows from the fact that: $Oj-1=Cj-1 \oplus Pj-1$

because: $Cj-1=Pj-1 \oplus Oj-1$

Therefore, the encryption output can be expressed in terms of the previous plaintext and ciphertext. **Demonstrating That Decryption Works**

By rearranging terms, we can demonstrate that decryption works.

Starting from: $C_j = P_j \oplus E(K, [C_{j-1} \oplus P_{j-1}])$

we rearrange the terms to obtain: $P_j = C_j \oplus E(K, [C_{j-1} \oplus P_{j-1}])$

This shows how the plaintext can be recovered.

Definition of OFB Mode

Initial value $I_1 = \text{Nonce}$

Input to subsequent encryption operations $I_j = O_{j-1}$

where: $j = 2, \ldots, N$. Thus, the previous encryption output becomes the next input.

Encryption Output

$O_j = E(K, I_j)$ where, $j = 1, \ldots, N$. Therefore, every input I_j is processed by the encryption function using key K.

Ciphertext Generation

The ciphertext is, $C_j = P_j \oplus O_j$

where: $j = 1, \ldots, N$

Thus:

Plaintext block
+
Encryption output
↓
XOR
↓
Ciphertext block

OFB Decryption

The initial value is: $I_1 = \text{Nonce}$

For subsequent blocks: $I_j = O_{j-1}$

where: $j = 2, \ldots, N$

The encryption function is then applied: $O_j = E(K, I_j)$

The plaintext is recovered by: $P_j = C_j \oplus O_j$

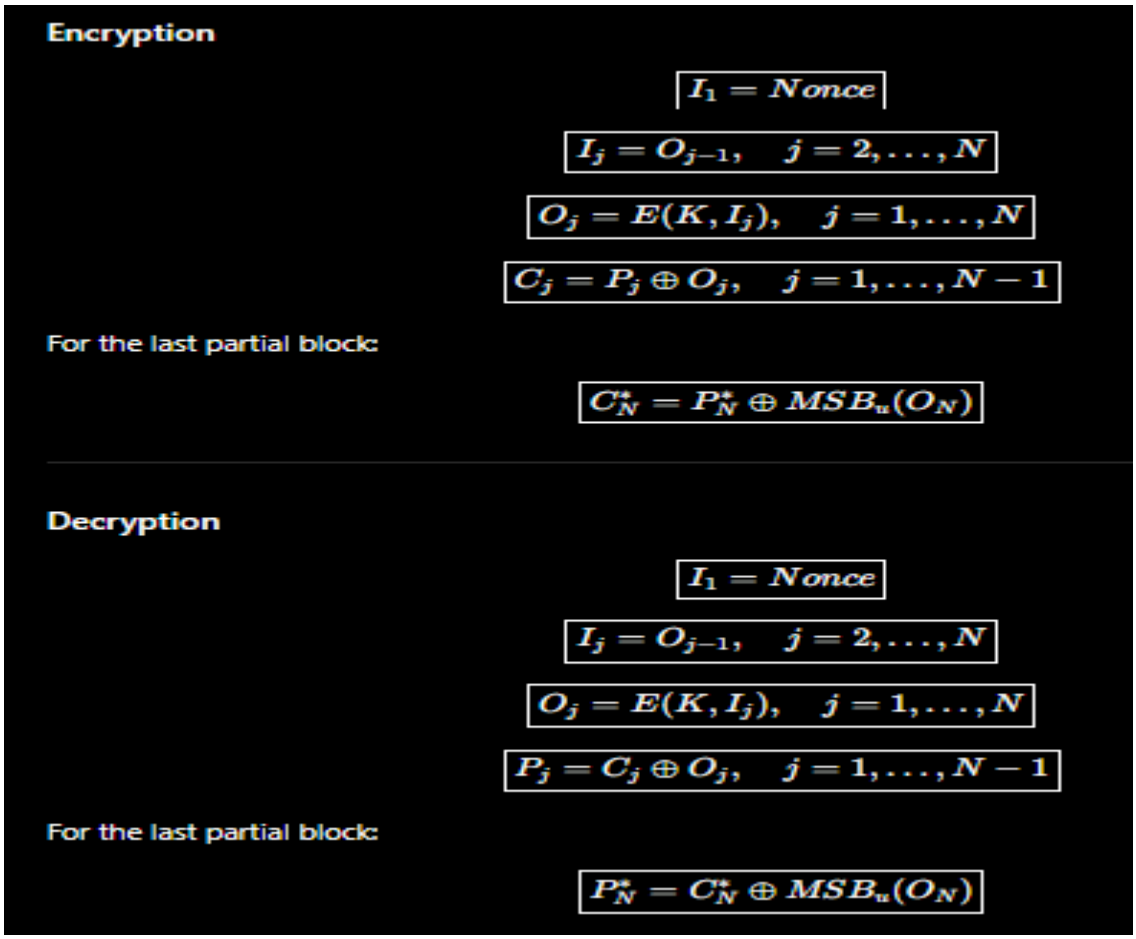
Why the Same Process Is Used for Decryption

During encryption: $C_j = P_j \oplus O_j$

During decryption: $P_j = C_j \oplus O_j$

This works because XOR is reversible. Thus, the same sequence of encryption outputs Oj is used for both encryption and decryption.

OFB Definition — Complete Equations



Last Partial Block-The case where the last plaintext block is not a complete block. Let the size of a block be **b**, Suppose the last plaintext block contains **u** bits. Where **u<b**. The **most significant** u **bits of the last output block ON** are used for the XOR operation.

Thus

$$C_N^* = P_N^* \oplus MSB_u(O_N)$$

The remaining, b−u bits of the last output block ON are discarded.

OFB Initialization Vector-As with **CBC and CFB**, OFB mode requires an initialization vector .However, in the case of OFB, the IV must be a **nonce**. That is, the IV must be unique to each execution of the encryption operation. The reason is related to the sequence of encryption output blocks Oi, depend only on,

- the key
- the IV

They do **not** depend on the plaintext. Therefore, for a given key and IV, the stream of output bits used to XOR with the plaintext bits is **fixed**. This is an important characteristic of OFB.

Fixed Output Stream-For a given K and IV the sequence O1,O2,O3,…is fixed. Therefore, the stream of output bits used for the XOR operation is also fixed. If two different messages have an **identical block of plaintext in the identical position**, an attacker would be able to determine that portion of the Oi stream.

Transmission Bit Errors- One advantage of OFB mode is that, Bit errors in transmission do not propagate.

For example, if a bit error occurs in Ci , only the recovered value of Pi is affected. The subsequent plaintext units are **not corrupted**.

In CFB, Ci also serves as input to the shift register. Therefore, an error in Ci causes **additional corruption downstream**. In OFB, the encryption output is fed back instead. Therefore, a transmission bit error does not propagate to subsequent plaintext units.

The disadvantage of OFB is that it is more vulnerable to a **message stream modification attack** than CFB. If a bit in the ciphertext is complemented, the corresponding bit in the recovered plaintext is also complemented. Therefore, modification of ciphertext can result in a corresponding controlled change in the recovered plaintext. Thus, controlled changes can be made to the recovered plaintext. This may make it possible for an opponent, by making the necessary changes to the **checksum portion of the message as well as the data portion**, to alter the ciphertext in such a way that the changes may not be detected by an **error-correcting code**.

OFB has the structure of a typical stream cipher.

This is because the cipher:

- takes an initial value and a key;
- generates a stream of bits as a function of the initial value and key;
- XORs the generated stream with the plaintext bits.

OFB encrypts plaintext **a full block at a time**. A typical block size can be 64 or 128 bits. Many stream ciphers, in contrast, encrypt **one byte at a time.**

COUNTER MODE

Interest in **Counter (CTR) mode** has increased recently because of applications to:

- **ATM (Asynchronous Transfer Mode)**
- **IPsec (IP security)**

The mode itself was proposed in **1979**. A counter having a size equal to the **plaintext block size** is used. The important requirement stated in **NIST SP 800-38A** is , The counter value must be different for each plaintext block that is encrypted.

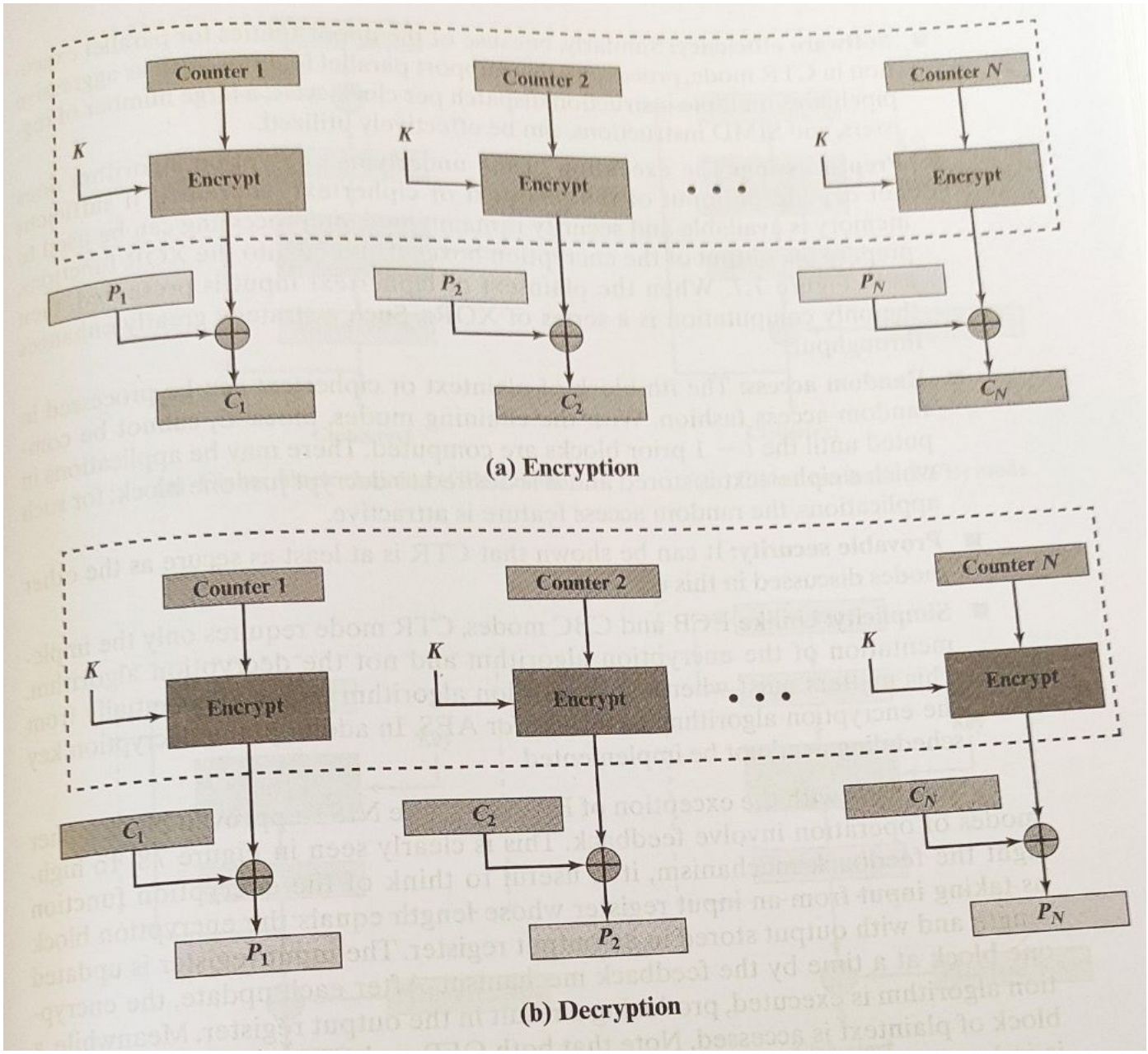
Counter Values:Typically, the counter is:

1. Initialized to some value.
2. Incremented by 1 for each subsequent block.

The increment is performed modulo 2^b .where b=block size. Thus, if the block size is b bits, the counter operates modulo 2^b. Suppose the counter values are T1,T2,…,TN. These are the counter values used for the plaintext blocks.Therefore
Counter 1 → Plaintext block P1

Counter 2 → Plaintext block P2
...
Counter N → Plaintext block PN

Each counter value is encrypted separately.



For encryption, the counter value is first encrypted using the key K.

For the first block: $E(K,T_1)$ is generated. This encrypted counter value is then XORed with the first plaintext block. Thus, $C_1=P_1 \oplus E(K,T_1)$

Second Block Encryption: For the second plaintext block, the next counter value is used. The counter value is T_2 It is encrypted $E(K,T_2)$. The result is XORed with P_2. Therefore, $C_2=P_2 \oplus E(K,T_2)$

The same process continues for all blocks. For block j: $Cj=Pj \oplus E(K,Tj)$

Thus: $C_1=P_1 \oplus E(K,T_1)$, $C_2=P_2 \oplus E(K,T_2)$, $C_3=P_3 \oplus E(K,T_3)$ and so on.

For encryption, the counter is encrypted and then XORed with the plaintext block to produce the ciphertext block, **there is no chaining**. This is an important characteristic of CTR mode. Each plaintext block is processed using its corresponding counter value.

For decryption, the **same sequence of counter values** is used. The encrypted counter value is generated again E(K,Tj).The resulting value is XORed with the ciphertext block. Thus: $P_j=C_j \oplus E(K,T_j)$

For the first block: $P_1=C_1 \oplus E(K,T_1)$ **.**The same counter value T_1 that was used during encryption must be available during decryption.

For the second block: $P_2=C_2 \oplus E(K,T_2)$ **.**Similarly, the counter value T_2 is encrypted using the same key.

For $j=1,\ldots,N-1$, $P_j=C_j \oplus E(K,T_j)$

Thus encryption and decryption use the same encrypted counter sequence.

Encryption

$$C_j = P_j \oplus E(K, T_j)$$

for:

$$j = 1, \ldots, N-1$$

For the final partial block:

$$C_N^* = P_N^* \oplus MSB_u[E(K, T_N)]$$

Decryption

$$P_j = C_j \oplus E(K, T_j)$$

for:

$$j = 1, \ldots, N-1$$

For the final partial block:

$$P_N^* = C_N^* \oplus MSB_u[E(K, T_N)]$$

- Pj= plaintext block
- Cj = ciphertext block
- Tj= counter value
- K= key
- E(K,Tj) = encryption of counter value Tjusing key K
- N = number of blocks
- P_N^* = final partial plaintext block
- C_N^* = final partial ciphertext block
- u = number of bits in the final partial block
- MSBu = most significant u bits

The case where the final plaintext block may be a **partial block of** u **bits**. For this final block u<b , where b is the block size. The **most significant** u **bits of the last output block** are used for the XOR operation. Thus,

$$C_N^* = P_N^* \oplus MSB_u[E(K, T_N)]$$

The remaining b−u bits are discarded.

Unlike the ECB, CBC, and CFB modes, we do not need to use padding because of the structure of the CTR mode. Therefore, when the last plaintext block is a partial block, the required number of bits is taken from the encrypted counter output. As with **OFB mode**, the initial counter value must be a **nonce**. That is T_1=Nonce. The initial counter value must be different for all messages encrypted using the same key.All Ti values across all messages must be unique. Therefore, the counter values must not be reused when the same key is being used. This is an important requirement of CTR mode.

If, contrary to the requirement, a counter value is used multiple times, then the confidentiality of all the plaintext blocks corresponding to that counter value may be compromised. Suppose a particular counter value is used more than once. For that counter value, the encryption output is E(K,Ti). The same key and same counter value produce the same encryption output. If the output of the encryption function for a given counter value is known, then the corresponding ciphertext block can be used to determine the output. Any other plaintext blocks that are encrypted using the same counter value to be easily recovered from their associated ciphertext blocks. Thus, counter reuse must be avoided.

Ensuring Counter Uniqueness-The counter value can be continuously incremented by 1 across messages. That is, the first counter value of each message is one more than the last counter value of the preceding message. Therefore,

Message 1: T_1, T_2, T_3, T_4 ...

Message 2: T_5, T_6, T_7, T_8 ...

Message 3: T_9, T_{10}, T_{11}, T_{12} ...

The counter values remain unique.

Advantages of CTR Mode:

Hardware Efficiency-Unlike the three **chaining modes**, encryption or decryption in CTR mode can be performed **in parallel on multiple blocks of plaintext or ciphertext**. For chaining modes, the algorithm must complete the computation on one block before beginning on the next block. This limits the maximum throughput of the algorithm. In CTR mode, the throughput is only limited by the amount of parallelism that is achieved.

Software Efficiency-CTR mode provides opportunities for parallel execution. Processors supporting parallel features such as:

- pipelining
- multiple instruction dispatch per clock cycle
- a large number of registers
- SIMD instructions

can be effectively utilized. Thus, the opportunities for parallel execution in CTR mode can be used efficiently by such processors.

Preprocessing-The execution of the underlying encryption algorithm does not depend on the input of the plaintext or ciphertext. Therefore, if sufficient memory is available and security is maintained, preprocessing can be used. The output of the encryption boxes can be prepared in advance. These outputs are then used as inputs to the XOR operation.

Preprocessing Process-Before the plaintext or ciphertext input is presented:

1. The encryption operations can be performed.
2. The outputs of the encryption boxes can be prepared.
3. These outputs can be stored.
4. When the plaintext or ciphertext becomes available, the required computation consists of a series of XOR operations.

Such a strategy can **greatly enhance throughput**.

Random Access-The ith block of plaintext or ciphertext can be processed in random-access fashion. This is possible because CTR mode does not use chaining between plaintext or ciphertext blocks.

Random Access With Chaining Modes-With chaining modes Block Ci cannot be computed until the i−1 prior blocks are computed.Therefore, access to a particular block requires processing the preceding blocks. CTR mode does not have this requirement.

Application of Random Access. There may be applications in which:

- a ciphertext is stored;
- it is desired to decrypt just one block.

For such applications the random-access feature is attractive.

Provable Security-It can be shown that CTR is at least as secure as the other modes.

Simplicity-Unlike **ECB and CBC** modes, CTR mode requires only the implementation of the encryption algorithm and not the decryption algorithm. This is important when the decryption algorithm differs substantially from the encryption algorithm. As it does for **AES** as an example of such a situation.

Key Scheduling-The decryption key scheduling need not be implemented. Thus, only the encryption operation is required for CTR mode.

Feedback in Block Cipher Modes-With the exception of **ECB**, all of the NIST-approved block cipher modes of operation involve **feedback**.

Feedback Mechanism-To understand the feedback mechanism, thinking of the encryption function as taking input from an **input register**. The input register has a length equal to the encryption block length.There is also an **output register**.The feedback mechanism updates the input register **one block at a time**.

Feedback Operation-The sequence described is:

1. The input register contains an input.
2. The encryption algorithm is executed.
3. The result is placed in the output register.
4. The feedback mechanism uses the result to update the input register.
5. The input register is then used for the next block.

This process continues one block at a time.

OFB and CTR-Both OFB and CTR produce output that is independent of both the plaintext and the ciphertext. Therefore, it states that OFB and CTR are natural candidates for stream ciphers that encrypt plaintext by XOR one full block at a time.

STREAM CIPHERS

Stream ciphers can be viewed as a pseudorandom equivalent of a **one-time pad**.

- A **one-time pad** uses a long random key equal in length to the plaintext message.
- A **stream cipher** uses a short secret key (K) and a pseudo randomly generated stream of bits (keystream Zi) that is computationally indistinguishable from a stream of random digits.

Traditionally, block ciphers were more widely used due to their ability to operate in different modes (such as Counter, OFB, and CBC). However, stream ciphers offer significant advantages:

- **High-Speed Streaming Data:** Well suited for encrypting continuous data feeds.
- **Constrained Devices:** Ideal for devices with very limited memory and processing power, such as small wireless sensors in the Internet of Things (IoT) and Radio Frequency Identification (RFID) tags.

Generic Structure of a Typical Stream Cipher

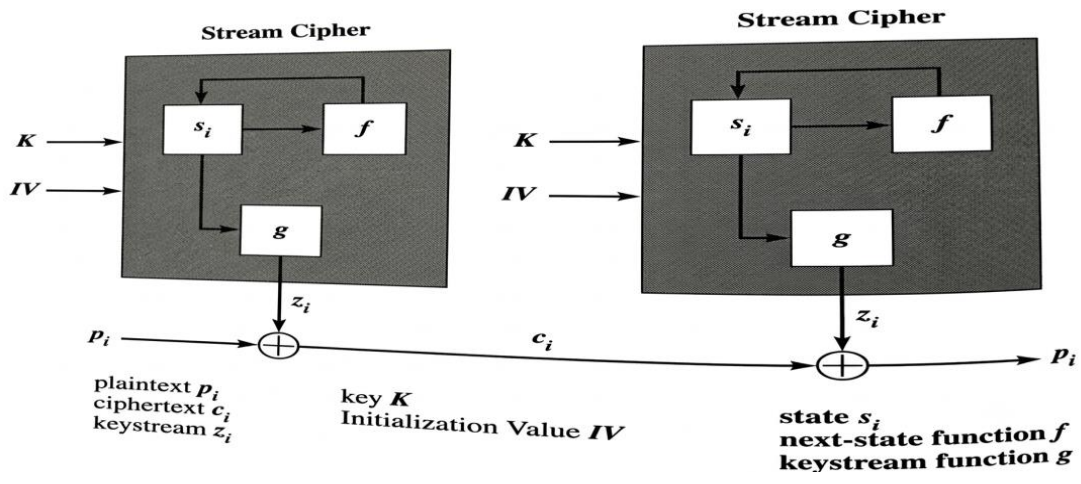
A typical stream cipher relies on three internal elements:

1. **Secret State (Si):** Internal memory state that evolves over time during encryption and decryption. The starting initial state is designated as S0.
2. **State Transition Function (f):** Computes a new state value from the old state value at each bit generation step.
3. **Output Function (g):** Produces the keystream bits (Zi) used for encryption and decryption.

Inputs:

- **Secret Key (K):** Initializes the state and may serve as an input parameter to f.
- **Initialization Vector (IV):** Used along with K to initialize the state. The IV does not need to be secret, but it **must be unpredictable and unique** for every execution.

Diagram: Generic Structure of a Typical Stream Cipher



Encryption & Decryption Logic

The keystream (Zi) is combined byte-by-byte (or bit-by-bit) with the plaintext (Pi) using the **bitwise exclusive-OR (XOR)** operation.

- **Encryption:** Ci = Pi XOR Zi

- **Decryption:** Pi = Ci XOR Zi

Example Step-by-Step:

Encryption:

11001100 Plaintext

XOR

01101100 Keystream

10100000 Ciphertext

Decryption:

10100000 Ciphertext

XOR

01101100 Keystream

11001100 Plaintext

Important Design Considerations:

Overview-When designing a pseudorandom number generator for a stream cipher, the goal is to create a keystream (Zi) that is computationally indistinguishable from a true random sequence. If the keystream exhibits predictable patterns, an attacker can break the cipher without needing the secret key. **The three critical design considerations are:**

1. Large Repetition Period (Maximum Period Length)-A pseudorandom number generator operates deterministically using finite internal memory (states). Because the number of possible states is finite, the state transition function (f) will eventually produce a state that occurred previously. From that point onward, the sequence of output bits will repeat endlessly in a deterministic loop.

Design Requirement:

- The generated keystream must have a **very large period** before it begins to repeat.
- **Vigenere Cipher Analogy:** In classical cryptography, the Vigenere cipher uses a repeating keyword. The shorter the keyword, the more frequently it repeats, making it easy for an attacker to determine the key length and break the cipher using frequency analysis. A longer keyword makes cryptanalysis significantly harder. Similarly, a stream cipher with a massive period prevents an attacker from observing repeating patterns in the keystream.

Cryptographic Impact:

- If the period is too short, an attacker capturing long ciphertext streams can align repeating sections of the keystream, effectively reducing the security to that of a simple polyalphabetic substitution cipher.
- For example, RC4 achieves a period length overwhelmingly likely to exceed 10^{100}, rendering period-repetition attacks computationally infeasible.

2. Pseudo-Randomness (Approximating True Randomness)-If the keystream contains statistical biases—such as generating more `1`s than `0`s, or favoring certain byte values—the resulting ciphertext will retain statistical traces of the original plaintext.

Design Requirement:

The keystream must approximate the mathematical properties of a **true random stream** as closely as possible:

- **Bit-Level Balance:** The stream must contain an approximately equal number of `0`s and `1`s (50% probability for each bit).
- **Byte-Level Uniformity:** When treating the keystream as a stream of 8-bit bytes, **all 256 possible byte values** (from `0x00` to `0xFF` or `0` to `255`) must appear with approximately equal frequency.
- **Unpredictability:** Knowing any prefix of N keystream bits should give an attacker no computational advantage in predicting the (N+1)-th bit.

Cryptographic Impact:

- The more statistically random the keystream appears, the more randomized and chaotic the resulting ciphertext becomes.
- High entropy (randomness) in the keystream prevents attackers from performing frequency analysis, differential attacks, or statistical guesswork.

3. Sufficient Key Length (Protection Against Brute-Force Attacks)-The output of the pseudorandom generator is conditioned directly on the secret input key (K). If the key space is too small, an attacker can test every possible key combination until the correct plaintext is recovered (exhaustive key search / brute-force attack).

Design Requirement:

- Just as with block ciphers, the secret key K used to initialize the internal state (S0) must be sufficiently long.
- **Modern Standard:** Under current technological capabilities, a key length of **at least 128 bits** (or up to 256 bits) is desirable to ensure strong security.

Cryptographic Impact:

- A 128-bit key size yields 2^{128} (approx. 3.4 × 10^{38}) possible key combinations.
- Testing every key would take billions of years using modern supercomputing clusters, rendering brute-force attacks mathematically impossible.

Stream Ciphers vs. Block Ciphers

Feature	**Stream Cipher**	**Block Cipher**
Key Reuse Risk	**Extreme Hazard:** If two plaintexts are encrypted with the same key/keystream, XORing the ciphertexts yields the XOR of the plaintexts (C1 XOR C2 = P1 XOR P2), exposing plaintext properties.	Keys can be safely reused across multiple blocks with proper modes of operation.
Speed & Code Size	Extremely fast; requires very few lines of code (e.g., RC4).	Slower without dedicated hardware acceleration (though modern hardware instructions like Intel AES-NI narrowed this gap).
Applications	Real-time streams over continuous channels, web browser links.	Fixed block data applications (file transfer, email, databases).

RC4

RC4 is a variable key-size stream cipher designed in 1987 by Ron Rivest for RSA Security. It uses byte-oriented operations based on a **random permutation**.

- **Period Length:** Overwhelmingly likely to be greater than 10^{100}.
- **Performance:** Executes in software very quickly (requires only 8 to 16 machine operations per output byte).
- **Applications:** Historically used in Wi-Fi Protected Access (WPA / IEEE 802.11), optional in SSH and Kerberos.

RC4 Structure and Operations

RC4 uses a **256-byte state vector S** containing elements S[0], S[1], ……., S[255], which holds a permutation of all 8-bit numbers from 0 through 255.

1. Initialization of Vector S and Temporary Vector T

Vector S is filled with values 0 to 255 in ascending order (S[0]=0, S[1]=1, ….., S[255]=255). Key K (length 1 to 256 bytes) is copied to vector T, repeating key bytes as necessary to fill T. The preliminary operations can be summarized as:

```
/* Initialization */
for i = 0 to 255 do
    S[i] = i;
    T[i] = K[i mod keylen];
```

2. Initial Permutation of S (Key-Scheduling Algorithm - KSA)

Vector T is used to permute vector S. This involves starting with S[0] and going through to S[255], and for each S[i],swapping S[i] with another byte in S according to a scheme dedicated by T[i]:

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    Swap(S[i], S[j]);
```

Because the only operation on S is a swap,the only effect is a permutation. S still contains all the numbers from 0 through 255.

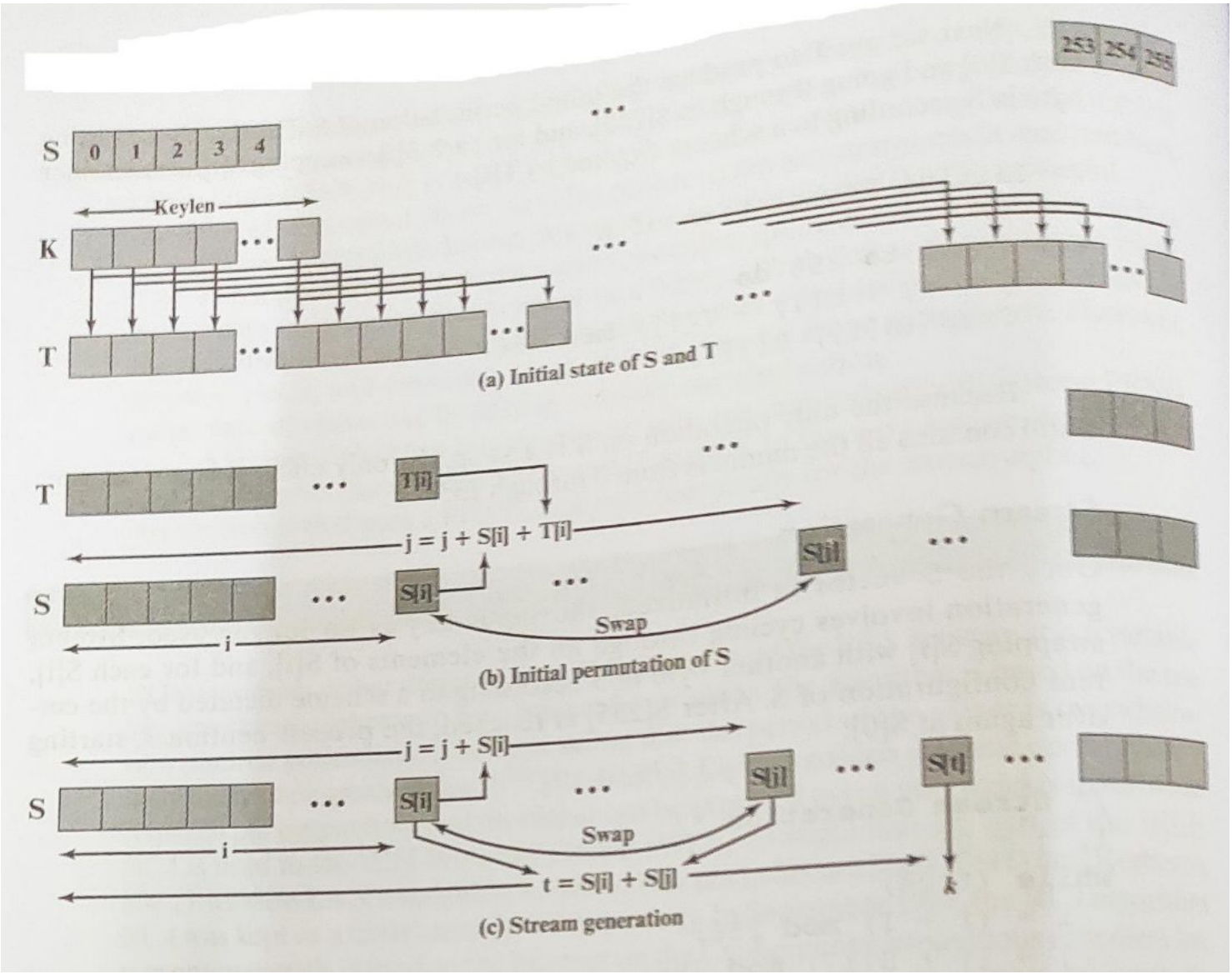
3. Stream Generation (Pseudo-Random Generation Algorithm - PRGA)

Once S is permuted, the original key is discarded. The keystream byte k is generated by continuously cycling through elements of S, swapping values, and picking the byte output k.

```
/* Stream Generation */
i = 0;
j = 0;
while (true) {
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap(S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
}
```

```
*To encrypt: XOR byte k with the next plaintext byte.
*To decrypt: XOR byte k with the next ciphertext byte.
```

Diagram: RC4 Logic



Strength & Cryptographic Weaknesses of RC4

- **KSA Weakness:** The key-scheduling algorithm contains non-randomness vulnerabilities, reducing effort to recover key bytes.
- **Keystream Biases:** Keystream output biases can be exploited to recover plaintexts encrypted multiple times.
- **Modern Deprecation:**
  - **IETF RFC 7465 (2015):** Prohibited RC4 in Transport Layer Security (TLS).
  - **NIST SP 800-52:** Prohibited RC4 for government implementations.